Predicting Kissat Performance with Satzilla

vorgelegt an der
Albert-Ludwigs-Universität Freiburg

von Rosas, Paulo
4141122

2. Mai 2026

universität freiburg

Bachelor's Thesis

Submitted on **May 2, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Paulo Rosas** (4141122)
Examined by **Prof. Dr. Mathias Fleury**

Computer Architecture
Department of Computer Science
Faculty of Engineering
Albert-Ludwigs-Universität Freiburg

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, May 2, 2026

Place, date	Paulo Rosas

Acknowledgements

Thanks to my supervisor Mathias Fleury.

Further thanks go to the corporate design team at the Albert-Ludwigs-Universität Freiburg and Leander Marius Bürkin for this LaTeX-template.

Contents

1 Einleitung **1**
1.1 Warum SAT wichtig ist . . . 2
1.2 Motivation . . . 3

2 Preliminaries **5**
2.1 Conjunctive Normal Form (CNF) . . . 5
2.2 Kissat . . . 6
2.2.1 Kissat Architektur . . . 6
2.2.2 Kissat-Konfigurationsvarianten . . . 9
2.2.3 Messung der Solver-Leistung . . . 9
2.3 SATzilla . . . 10
2.4 Funktionsweise . . . 10
2.4.1 Feature Extraction . . . 11
2.4.2 Machine Learning Model . . . 11
2.4.3 Training und Prediction Pipeline . . . 12

3 Satzilla Outcome **13**
3.1 Validierung mit Fake Solvern . . . 13
3.1.1 Design der fake Solver . . . 13
3.1.2 Resultate . . . 14
3.1.3 Interpretation . . . 14
3.2 Trainieren auf echten Kissat Solver Daten . . . 16
3.2.1 Dataset . . . 16
3.2.2 Training Konfiguration . . . 16
3.2.3 Testgenauigkeit . . . 16
3.3 Top-K-Genauigkeit: Der Wert des „fast Richtigen“ . . . 18
3.3.1 Resultate . . . 18
3.3.2 Interpretation – Zeitlicher Vorteil gegenüber dem Standard-Solver 19
3.4 Feature Importance Analysis . . . 21
3.4.1 Wichtigste Features . . . 21
3.4.2 Was die Features tatsächlich messen . . . 22
3.4.3 Feature Kategorien – Zusammenfassung . . . 26
3.5 Per-Solver Performance . . . 27
3.5.1 Interpretation . . . 28
3.6 Auswirkungen der Einbindung von LP-Features . . . 29
3.6.1 Ergebnis . . . 29
3.6.2 Interpretation . . . 29
3.7 Zusammenfassung der wichtigsten Ergebnisse . . . 30
3.8 Fazit . . . 31

4 Kissat Results **32**
4.1 Setup 34
4.1.1 Feature Extraction Process 34
4.1.2 SATZilla Model Training 35
4.2 Auswirkungen der Vorverarbeitung auf die Vorhersagen von SATZilla . . 37
4.2.1 Veränderungen der Vorhersageverteilung 37
4.2.2 Änderungen der Vorhersagen nach Vorverarbeitungsstufe 38
4.2.3 Analyse der Solver-Migration 40
4.3 Abschließende Schlussfolgerung: Was uns diese Ergebnisse sagen . . 42
4.3.1 Fazit 43
4.4 Vergleich der Featuresbedeutung für Pre10k und Pre100k 45
4.4.1 Wichtigste Features für jede Vorverarbeitungsstufe 45
4.4.2 Fazit 47
4.5 Laufzeitvergleich Pre10k vs. Pre100k 48
4.5.1 Ergebnisse Pre10k (10.000 Simplifikationsschritte) 49
4.5.2 Ergebnisse Pre100k (100.000 Simplifikationsschritte) 51
4.5.3 Wichtige Erkenntnisse 52
4.5.4 Interpretation 53
4.5.5 Fazit 53

5 Conclusion **54**
5.1 Technische Daten 55

List of Figures **II**

List of Tables **III**

References **V**

1 Einleitung

Das Erfüllbarkeitsproblem der Aussagenlogik (SAT, von englisch satisfabiality "Erfüllbarkeit") ist ein fundamentales Problem der Informatik mit kritischen Anwendungen in Software und Hardware Überprüfung, im automatisierten Testen, in der Planung, in der Bioinformatik sowie in vielen weiteren Bereichen der Computer-Aided Design und der Künstlichen Intelligenz.

Aufgrund seiner zentralen Bedeutung für die Komplexitätstheorie – SAT war das erste nachgewiesene NP-vollständige Problem – dient es als Ausgangspunkt für die Analyse unzähliger weiterer algorithmischer Fragestellungen. Die typische Eingabe für SAT-Solver ist eine aussagenlogische Formel in konjunktiver Normalform (KNF), also eine Konjunktion von Klauseln, wobei jede Klausel eine Disjunktion von Literalen (Variablen oder negierten Variablen) darstellt.

Die Frage, ob eine solche Formel erfüllbar ist – das heißt, ob es eine Belegung der booleschen Variablen mit den Werten wahr oder falsch gibt, die die gesamte Formel wahr macht – ist entscheidungsrelevant: Erfüllbare Formeln belegen die Korrektheit eines Modells, während unerfüllbare Formeln beweisen, dass keine solche Belegung existiert.

Fortschritte in der SAT-Lösungstechnologie, insbesondere durch das CDCL (Conflict-Driven Clause Learning)-Verfahren, haben es ermöglicht, Instanzen mit Millionen von Variablen und Klauseln effizient zu lösen. Dadurch ist SAT zu einem praktischen Werkzeug für viele reale Anwendungen geworden, bei denen komplexe logische Bedingungen geprüft werden müssen.

1.1 Warum SAT wichtig ist

Trotz seiner theoretischen Bedeutung als erstes NP-vollständiges Problem (Cook, 1971) hat SAT breite praktische Anwendung in den Bereichen Hardware-Verifikation, Software-Testing, Planung und künstliche Intelligenz gefunden.[2]

Moderne SAT-solver haben in den letzten zwei Jahrzenten bemerkenswerte fortschritte gemacht; Löser wie Kissat (Biere et al., 2020) sind in der Lage, industrielle Instanzen mit Millionen von Variablen zu lösen.[1]

SAT hat breite praktische Anwendung gefunden, da sich viele Probleme aus der realen Welt als SAT-Instanzen formulieren lassen:

- **Hardware-Verifikation**: Nachweis, dass sich digitale Schaltungen korrekt verhalten
- **Softwaretests**: Erstellung von Testfällen, die bestimmte Pfade abdecken
- **Planung und Terminierung**: Ermittlung von Handlungsabläufen, mit denen Ziele erreicht werden
- **Kryptoanalyse**: Knacken kryptografischer Systeme durch das Lösen logischer Beschränkungen
- **Bioinformatik**: Analyse genetischer Sequenzen und Proteinfaltung

Die praktische Bedeutung von SAT hat jahrzehntelange Forschungen zur Entwicklung effizienter SAT-Löser vorangetrieben.

Der praktische Erfolg der SAT-Lösung ist weitgehend auf die bemerkenswerte Leistungsfähigkeit von CDCL-Lösern (Conflict-Driven Clause Learning) zurückzuführen. In der SAT-Forschung ist jedoch bekannt, dass kein einzelner Löser bei allen Instanzen allen anderen überlegen ist. Verschiedene Solver weisen komplementäre Leistungsmerkmale auf – ein Solver, der bei einer Instanzfamilie extrem schnell ist, kann bei einer anderen schlecht abschneiden. Diese Beobachtung hat zur Entwicklung von Algorithmusauswahlsystemen geführt, die automatisch den für eine bestimmte Instanz am besten geeigneten Solver auswählen.

Diese Erkenntnis hat zur Entwicklung des portfoliabasierten SAT-Lösens geführt, bei dem eine Sammlung von Solvern gepflegt wird und ein Auswahlmechanismus entscheidet, welcher Solver für eine bestimmte Instanz eingesetzt wird. Die zugrunde liegende Idee ist einfach, aber wirkungsvoll: Wenn wir vorhersagen können, welcher Solver bei einer Instanz die beste Leistung erbringt, können wir erhebliche Rechenzeit einsparen,

indem wir Solver mit schlechter Leistung vermeiden.

1.2 Motivation

In meinem ürsprunglichen Bachelorprojekt, was mit dieser Bachelorarbeit verknüpft ist, bestannt meine Aufgabe darin Satzilla im jahr 2025 zu wiederholen mit neuen Daten um dann zu vergleichen wie diese neuen vorhersagen mit den Alten aus dem Jahr 2024 übereinstimmen würden.

Dafür bekam ich von meinem Tutor Dr. Mathias Fleury den code von Satilla des Github repos und 400 cnf Dateien aus der enstprechenden webseite.

Die Arbeit war nicht immer einfach und ich hatte mit technischen Problemen zu kämpfen, so fehlten gewisse Dateien für die ich an dem Autor der Universität Aachen von Satzilla anschreiben musste. Des weiteren kam mein eigener PC an seine grenzen da er für die benötigten Vorhersagen nicht genug RAM hatte.

Dank der hilfe meiner beiden Tutoren konnte und der Bereitstellung eines PCs meiner Universität mit dem ich mich mit einer SSH Verbindung anmelden und benutzen konnte, konnte ich mein Arbeit zuende vollstellen.

Auch wenn das ergebniss meiner Arbeit nicht komplett zufriedenstellend war fand ich die Arbeit und Forschung interresant und herausfordernt.

Besonders interresiert hat mich, welche die Predictions waren die mir Satzilla jeweils für meine benchmarks empfohlen hat mit der dazugehörigen Laufzeit. Diese Daten konnte ich dann auswerten.

Für meine Bachelorarbeit, Predicting Kissat Performaance with Satzilla, wurde mir dann die volgenden zwei themen zugeweiesen:

Erstes habe ich getestet ob man Satzilla komplett auf gewissen Kissat Konfigurationen trainiren kann, sprich ob sie korrekte predicitons machen. Dafür musste ich als erstes auch testen ob Satzilla überhaupt mit seinem Machine Learning funktioniert da die ergebnisse im meinem Bachelorprojekt zweifel drauf ankommen ließen.

Zweitens habe ich getesten zu welchen ergebnissen man kommt wenn man Satzilla mit kissat trainiert und dann auf preprocessed daten anwendet, einaml die benchmarks

auf pre10k vereinfacht und dann auf pre100k vereinfacht, was bis dato niemand zuvor gemacht hat.

2 Preliminaries

In diesem Kapitel beschreibe ich die Grundlagen die für die Späteren Kapitel von bedeutung sein werden. Zuerst was CNF Dateien sind un wie sie aufgebaut sind, dann was Kissat und Satzilla sind und wie sie mit Dateien trainiert werden um Predicitons von Solvern zu machen.

2.1 Conjunctive Normal Form (CNF)

Bei einer Boolschen Formel, die aus durch UND- und ODER-Operatoren verbunden Variablen besteht, (AND $\wedge$, OR $\vee$, NOT $\neg$) geht es beim SAT-Problem um die Frage, ob es eine Zuorndung von Warheitswerten zu den Variablen gibt, bei der die gesamte Formel als Wahr ausgewertet wird.

Moderne SAT-Löser arbeiten in der Regel mit Formeln in konjunktiver Normalform (CNF). Eine CNF-Formel besteht aus:

- **Variablen**: Boolesche Variablen (z. B. x_1, x_2, ..., x_n)
- **Literale**: Eine Variable oder deren Negation (z. B. x_1 oder $\neg x_1$)
- **Klauseln**: Eine Disjunktion (ODER) von Literalen (z. B. $(x_1 \vee \neg x_2 \vee x_3)$)
- **Formel**: Eine Konjunktion (UND) von Satzteilen (z. B. $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$

Das CNF-Format ist sowohl ausdrucksstark (jede Boolesche Formel lässt sich in CNF umwandeln) als auch für algorithmisches Denken gut geeignet. Das folgende Beispiel zeigt eine einfache CNF-Datei:

```
p cnf 3 4
1 -2 0
-1 3 0
2 -3 0
-1 -2 -3 0
```

Dies stellt eine Formel mit 3 Variablen und 4 Klauseln dar. Die Zeile „p cnf“ gibt den Problemtyp und die Dimensionen an (3 Variablen, 4 Klauseln), und jede nachfolgende Zeile stellt eine Klausel dar, die mit 0 endet.

2.2 Kissat

Kissat ist ein hochmoderner SAT-Löser nach dem Prinzip des Conflict-Driven Clause Learning (CDCL), der von Armin Biere und seiner Forschungsgruppe an der Johannes Kepler Universität Linz entwickelt wurde. Der Kissat-SAT-Solver ist eine komprimierte und verbesserte Neuimplementierung von CaDiCaL in C. Er verfügt über verbesserte Datenstrukturen, eine optimierte Ablaufplanung der Datenverarbeitung sowie optimierte Algorithmen und Implementierungen.

Kissat belegte den ersten Platz in der Hauptkategorie des SAT-Wettbewerbs 2020 und den ersten Platz bei den unlösbaren Instanzen.[3]

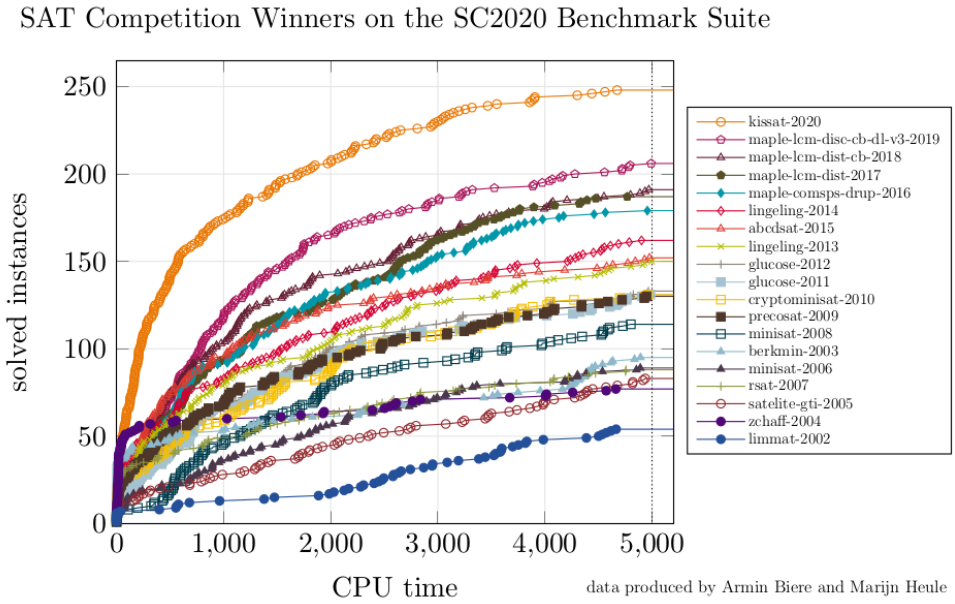


Figure 1: Kissat SAT Solver [3]

2.2.1 Kissat Architektur

Kissat implementiert den CDCL-Algorithmus mit mehreren ausgefeilten Optimierungen:

Kissat ist ein „einfacher und übersichtlicher Bare-Metal-SAT-Löser“, der in C geschrieben ist. Es handelt sich um eine Portierung von CaDiCaL zurück nach C mit verbesserten Datenstrukturen, einer optimierten Ablaufplanung der Datenverarbeitung sowie optimierten Algorithmen und Implementierungen.

1. Variblen Elimination

Die Variableneliminierung ist eine Vorverarbeitungstechnik, die eine SAT-Formel vere-

infacht, indem eine Variable und alle Klauseln, die diese enthalten, entfernt werden, während neue Klauseln hinzugefügt werden, die die logische Äquivalenz bewahren.

Die Eliminierung von x ersetzt alle Klauseln, die x enthalten, durch die **Resolventen** zwischen jeder positiven und jeder negativen Klausel:

$$(x \vee A) \wedge (\neg x \vee B) \implies (A \vee B)$$

Die Variable x wird dann aus der Formel entfernt und die neuen Resolventen-Klauseln werden hinzugefügt.

Das ist in dieser Arbeit die wichtigste eigenschaft da weniger Variablen bedeutet eine geringerere Verzweigungsfaktor im CDCL-Suchbaum und durch diese simplifikation kann die Lösung des Problems verschnellert werden.

Das Vorhandensein oder Fehlen der Variableneliminierung ist einer der Hauptunterschiede zwischen den Kissat-Konfigurationen, weshalb das Feature „reducedVars“ in unserem Satzilla-Modell als äußerst wichtiges Merkmal auftritt (Figure 7).

2. Subsumption Elimination

Durch Subsumption werden Klauseln entfernt, die sich logisch aus anderen, kürzeren Klauseln ergeben.

Beispiel: Die Klausel $(x \vee y)$ subsumiert $(x \vee y \vee z)$, da letztere immer dann wahr ist, wenn erstere wahr ist.

3. Probing (Failed Literal Propagation)

Untersuchung von Versuchen, Literale zuzuweisen und erzwungene Zuweisungen oder Unerfüllbarkeit zu erkennen.

4. Caching (Learned Clause Management)

Kissat speichert Klauseln und verwendet sie nach einem Neustart erneut, um doppelte

Arbeit zu vermeiden.

5. Multi-Level Decision Heuristic

Die Entscheidungsheuristik legt fest, auf welche Variable und welche Phase als Nächstes verzweigt wird.

6. Restart Strategy

Bei einem Neustart wird der aktuelle Suchzweig verlassen und der Entscheidungsprozess neu gestartet, wobei die erlernten Klauseln beibehalten werden.

7. Inprocessing (Intermediate Simplification)

Inprocessing wendet Vereinfachungstechniken während der gesamten Lösung wiederholt an, nicht nur zu Beginn.

2.2.2 Kissat-Konfigurationsvarianten

Diese sechs Kissat-Varianten decken die wichtigsten Dimensionen des Konfigurationsraums ab – von minimaler Vorverarbeitung (noeliminate) über Standardverhalten (default) bis hin zu maximaler Stabilisierung (stable, stable2) und alternativen Heuristiken (target2) – und ermöglichen es Satzilla somit, für jede Instanz die passendste Strategie zu erlernen.

Dies ist der Grund weshalb genau diese sechs Kissat Solver für diese Arbeit ausgewählt, da sie am relevantesten sind.

Kissat bietet zahlreiche Konfigurationsoptionen, mit denen sich sein Verhalten anpassen lässt. Die in dieser Arbeit verwendeten Varianten stellen unterschiedliche Kompromisse zwischen dem Aufwand für die Vorverarbeitung und der Rechengeschwindigkeit dar:

Variante	**Haupteigenschaften**
kissat_default	Standardkonfiguration mit ausgewogenen Einstellungen
kissat_noeliminate	Deaktiviert die Variableneliminierung
kissat_nostable	Deaktiviert den stabilen Modus
kissat_stable	Aktiviert den stabilen Modus für eine Suche
kissat_stable2	Erweiterter Stabilitätsmodus mit zusätzlichen Heuristiken
kissat_target2	Optimiert für eine bestimmte Problemklasse

Table 1: Kissat-Konfigurationsvarianten für diese Arbeit

2.2.3 Messung der Solver-Leistung

Die Leistung eines Solvers wird in der Regel anhand der Gesamtlaufzeit oder der CPU-Zeit gemessen. In dieser Arbeit verwende ich die vom Feld „process-time“ von Kissat gemeldete Laufzeit, die die zur Lösung der Instanz aufgewendeten CPU-Sekunden angibt. Zeitüberschreitungen (z. B. 3600 Sekunden, 7200 Sekunden) deuten darauf hin, dass der Solver die Berechnung nicht innerhalb der zugewiesenen Zeit abgeschlossen hat. Daraus schließt man das das Problem unsatisfiable ist, sprich der solver konnte dafür keine Lösung finden die das Problem erfüllen könnte.

2.3 SATzilla

SATZilla ist ein Algorithmus-Portfolio für SAT-Probleme, das auf sogenannten "Empirical Hardness Models" basiert – lernenden Modellen, die die Laufzeit eines gegebenen Algorithmus auf einer gegebenen SAT-Instanz vorhersagen. [4]

Es ist seit langem bekannt, dass es keinen einzelnen "dominanten" SAT-Solver gibt – verschiedene Solver arbeiten auf verschiedenen Instanzen unterschiedlich gut . Anstatt für eine ganze Klasse von Instanzen einen einzigen Solver zu wählen, trifft SATzilla diese Entscheidung online, auf Einzelinstanz-Basis.[5]

2.4 Funktionsweise

SATzilla verwendet ein zweistufiges Verfahren:

1. **Merkmalsextraktion**: Aus einer gegebenen SAT-Instanz werden strukturelle Eigenschaften extrahiert (z.B. Anzahl der Variablen und Klauseln, Graph-Eigenschaften).

2. **Laufzeitvorhersage**: Basierend auf diesen Merkmalen sagt ein gelerntes Modell voraus, welcher Solver voraussichtlich am schnellsten sein wird. Die Architektur enthält Module für Feature Extraction, Runtime Prediction und Portfolio Selection.

Die Methode ist weitgehend automatisiert: Sie erhält als Eingabe eine Verteilung von Probleminstanzen und eine Menge von Kandidaten-Solvern und konstruiert ein Portfolio, das eine bestimmte Zielfunktion optimiert.

SATzilla ist ein auf **maschinellem Lernen basierendes Portfolio-System**, das für jede neue SAT-Instanz den voraussichtlich schnellsten Solver aus einer Menge von Kandidaten auswählt. Die Methode nutzt extrahierte Merkmale der Instanz, um die Laufzeit verschiedener Solver vorherzusagen, und hat ihre Überlegenheit gegenüber einzelnen Solvern in mehreren SAT Competitions bewiesen.

2.4.1 Feature Extraction

Satzilla extrahiert aus jeder CNF-Formel über 100 Merkmale, die in mehrere Kategorien unterteilt sind:

Kategorie	Features	Was sie messen
Basic	Anzahl der Variablen, Anzahl der Klauseln, Verhältnis	Größe und Dichte
Variable-Clause Graph (VCG)	Mittelwert, Entropie, Min/-Max	Verbindungsstruktur eines Graphen
Clause Graph (CG)	Klauselverbindungen, Cluster	Wechselwirkungen zwischen Klauseln
Variable Graph (VG)	Variable co-occurrence	Variable Beziehungen
Horn Features	Horn-Klausel-Bruch, Eigenschaften der Horn-Variablen	Besondere Struktur
Probing Features	Ausbreitungstiefe der Einheit, reduzierte Variablen	Eigenschaften des Suchraums
Local Search	Leistungskennzahlen von GSAT/SAPS	Die Landschaft der lokalen Suche
LP Features	Relaxationswerte der linearen Programmierung	Kontinuierliche Entspannung

Table 2: SAT Instance Features Categories

Diese Merkmale werden von einem speziellen Merkmalsextraktor (Merkmalsexekutivdatei) berechnet, der die CNF-Datei analysiert und eine CSV-Datei ausgibt, die eine Zeile pro Instanz enthält.

2.4.2 Machine Learning Model

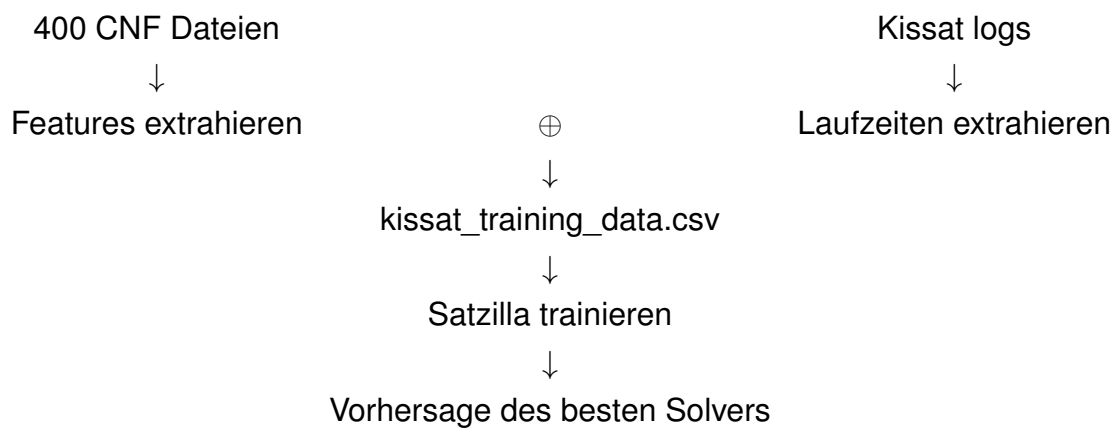
Satzilla nutzt „Random Forest“ (Breiman, 2001) als Lernalgorithmus. „Random Forest“ ist ein Ensemble-Verfahren, bei dem mehrere Entscheidungsbäume erstellt und deren Vorhersagen zusammengefasst werden. Wesentliche Vorteile für diese Anwendung:

- Unterstützt gemischte Merkmalstypen (kontinuierlich, kategorial)
- Unempfindlich gegenüber irrelevanten Merkmalen (der Algorithmus ignoriert verrauschte Eingaben)
- Zeigt die Merkmalsbedeutung an (welche Merkmale am wichtigsten sind)
- Verhindert Überanpassung durch geeignete Hyperparameter

Im ursprünglichen Satzilla sagt das Modell die Laufzeit direkt voraus. Für die Solverauswahl müssen wir jedoch nur wissen, welcher Solver am schnellsten ist – nicht die genaue Laufzeit. Diese Arbeit konzentriert sich auf die Genauigkeit der Solverauswahl.

2.4.3 Training und Prediction Pipeline

Die gesamte Satzilla-Pipeline besteht aus:



Mithilfe des SAT-Features-Extraktors habe ich 139 Features aus 400 CNF-Benchmarks extrahiert. Unabhängig davon habe ich die Protokolle des Kissat-Lösers ausgewertet, um die genauen Laufzeiten für sechs Lösungsvarianten bei denselben Instanzen zu ermitteln.

Nachdem ich die Merkmals- und Laufzeitdaten über eine Namenszuordnung abgeglichen hatten, identifizierte ich dann den schnellsten Solver pro Instanz und trainierten einen Random-Forest-Klassifikator. Daraus entstand ein Satzilla-Modell, das anhand der Merkmale einer neuen Instanz vorhersagt, welcher Kissat-Solver die beste Leistung erbringen wird.

3 Satzilla Outcome

In diesem Kapitel beschreibe ich wie ich Satzilla mit sogenanten Fake Solvern valediert habe und anschließend wie ich Satzilla mit kissat trainiert habe.

Zum schluss beschreibe ich die Features und ihre Wichtigkeit die ich durch das Training von Satzilla bekommen habe.

3.1 Validierung mit Fake Solvern

Da ich in meinem ursprünglichen Bachelorprojekt Predictions in Satzilla hatte die nicht mit den tatsächlichen vorhersagen übereinstimmten (so wurde fälschlicherweiße ISAsat alle der beste Solver für viele Probleme empfohlen) riet mir mein Tutor Dr. Mathias Fleury zu testen ob Satzilla überhaupt richtig funktioniert und passende vorhersagen machen kann.

Bevor ich Satzilla auf die Laufzeitdaten realer Solver angewandt haben, habe ich zunächst die gesamte Pipeline anhand künstlich erzeugter Solver-Verhaltensmuster validiert. Dieser Schritt war entscheidend, um sicherzustellen, dass die Merkmalsextraktion, die Datenausrichtung und der Lernmechanismus korrekt funktionieren.

3.1.1 Design der fake Solver

Ich habe künstliche Solver erstellt, die explizite, bekannte Beziehungen zu den Eigenschaften der Instanzen aufweisen (z. B. zur Anzahl der Variablen). Auf diese Weise konnte ich eine Referenzgröße definieren: Für jede Instanz wussten ich genau, welcher Solver als der schnellste anzusehen war.

- **3-Solver-Test**: Auf kleine, mittlere und große Instanzen spezialisierte Solver, basierend auf Schwellenwerten für die Variablenanzahl (33. und 67. Perzentil).
- **4-Solver-Test**: Zwei Solver basieren auf der Variablenanzahl und zwei auf der Klauselanzahl, wobei eine klare Hierarchie besteht (Variablenspezialisten dominieren).

3.1.2 Resultate

Genauigkeit der Ergebnisse der Fake Solver:

Test	Solver Count	Accuracy
Sprungfunktion mit beliebigen Schwellenwerten (100k/200k)	3	65%
Lineare Funktion (laufzeit in relation zur Anzahl der Variablen)	3	99%
Sprungfunktion mit datenabhängigen Schwellenwerten	3	99.25%
Schrittweise Hierarchie (Variablen + Klauseln)	4	99.75%

Table 3: Testergebnisse für verschiedene Auswahlstrategien

Eine Genauigkeit von 99 % bedeutet, dass Satzilla in 99 von 100 Fällen den schnellsten Fake Solver korrekt identifiziert, was beweist, dass die Merkmalsextraktion und das Training einwandfrei funktionieren.

Wichtigste Features der Fake Solver:

Rang	Feature	Relevanz
1	nvarsOrig	13.07%
2	VCG-CLAUSE-min	8.46%
3	nvars	7.79%

Table 4: Testergebnisse der wichtigsten Features

Je höher der Prozentsatz des Features ist, desto stärker stützt sich das Modell bei seinen Vorhersagen auf dieses Feature.

3.1.3 Interpretation

Die hohe Genauigkeit ($\geq 99\%$) in drei von vier Tests und die Auswahl der Features bestätigte, dass:

- Die Feature extraction erfasst die Eigenschaften der Instanzen korrekt. nvarsOrig führt die direkte Messung der Anzahl der ursprünglichen Variablen durch – das wichtigste Unterscheidungsmerkmal, nvars die Anzahl der Variablen nach dem Preprocesing und VCG-CLAUSE-min den minimaler Klauselgrad in einem Graphen mit variablen Klauseln (korreliert mit der Größe)
- Die Trainingspipeline (Random-Forest-Klassifikator, Kreuzvalidierung) funktioniert wie vorgesehen.

- Das Scheitern des ersten Tests (65 %) war auf schlecht gewählte Schwellenwerte (100k/200k) zurückzuführen, die zu einem unausgewogenen Datensatz führten – nicht auf einen Fehler in Satzilla selbst.

Fazit: Die Satzilla-Pipeline funktioniert einwandfrei und ist bereit für das Training mit echten Solver-Daten.

3.2 Trainieren auf echten Kissat Solver Daten

Nach der Validierung habe ich Satzilla mit sechs Konfigurationen des Kissat-Solvers trainiert:

- kissat_default
- kissat_noeliminate
- kissat_nostable
- kissat_stable
- kissat_stable2
- kissat_target2

3.2.1 Dataset

- **397 Instanzen** mit vollständigen Features, die Features wurden mithilfen des Satzilla tools extrahiert
- **Laufzeiten** für jeden Solver bei jeder Instanz (aus tatsächlichen Ausführungen)
- **Train/test split**: 80 % Training (317 Fälle), 20 % Test (80 Fälle), stratifiziert, um die Verteilung der Solver beizubehalten

3.2.2 Training Konfiguration

- **Model**: Random Forest Classifier (200 trees, max depth 20)
- **Features**: 131 strukturelle Features (ohne LP-Merkmale, gemäß rt_pred.py)
- **Ziel**: Für jede Instanz der Solver mit der kürzesten Laufzeit (der schnellste)

3.2.3 Testgenauigkeit

Was wir in dieser Graphik sehen ist das in 43.8% der Fälle Satzilla trainier auf Kissat tatstächlich den schnellsten und somit besten Solver empfiehlt.

Dies zeigt uns das selbst bei begrenzten Trainingsdaten (397 Instanzen für 6 Solver) schneidet Satzilla deutlich besser ab als eine zufällige Auswahl.

Somit ist die Auswahl von Satzilla für den schnellsten solver um 162% besser als einen der sechs Solver durch zufallsprinzip auszusuchen.

Satzilla trainiert auf Kissat zeigt das es zuverlässig den schnellsten solver empfehlen kann.

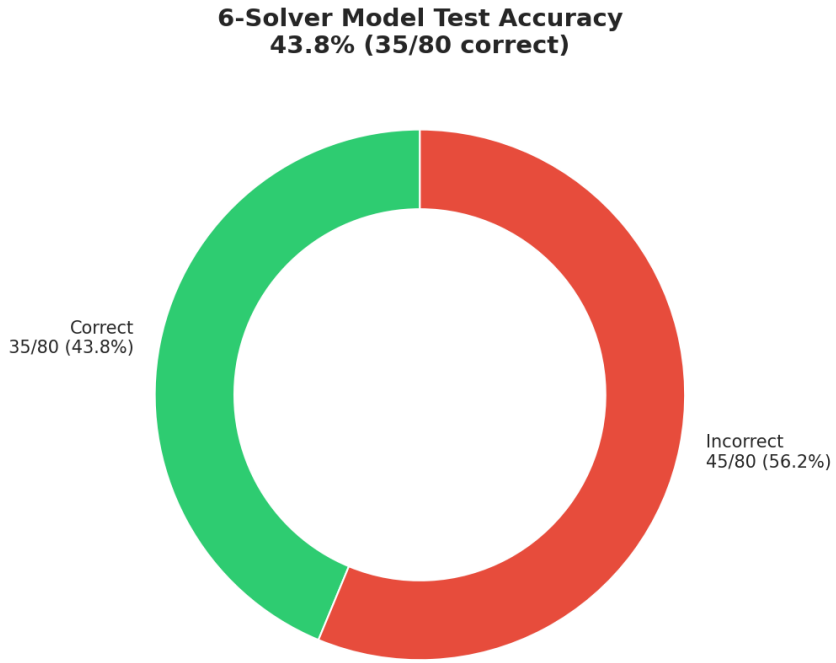


Figure 2: Testgenauigkeit der verschiedenen Strategien

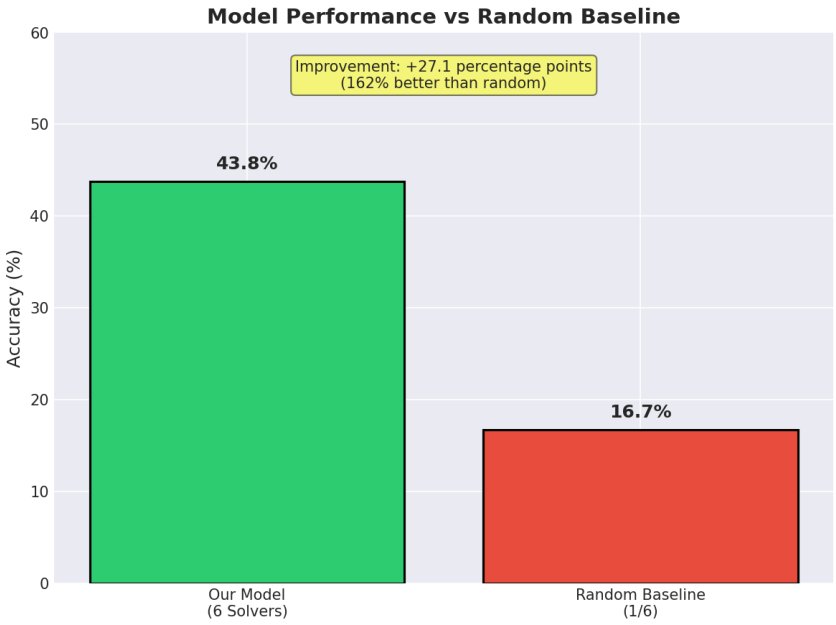


Figure 3: Vergleich mit Random Selection

3.3 Top-K-Genauigkeit: Der Wert des „fast Richtigen“

Bei der Lösung von SAT-Problemen in der Praxis reicht es oft aus, wenn das Ergebnis nahe am optimalen Lösungsvorschlag liegt. Ein Nutzer kann die ersten k Empfehlungen nacheinander ausprobieren. Daher habe ich auch die Genauigkeit der Top-2- und Top-3-Ergebnisse gemessen – also die Wahrscheinlichkeit, dass der tatsächlich schnellste Lösungsvorschlag unter den ersten k Vorhersagen auftaucht.

3.3.1 Resultate

Hier die Ergebnisse dargestellt als Grafiken

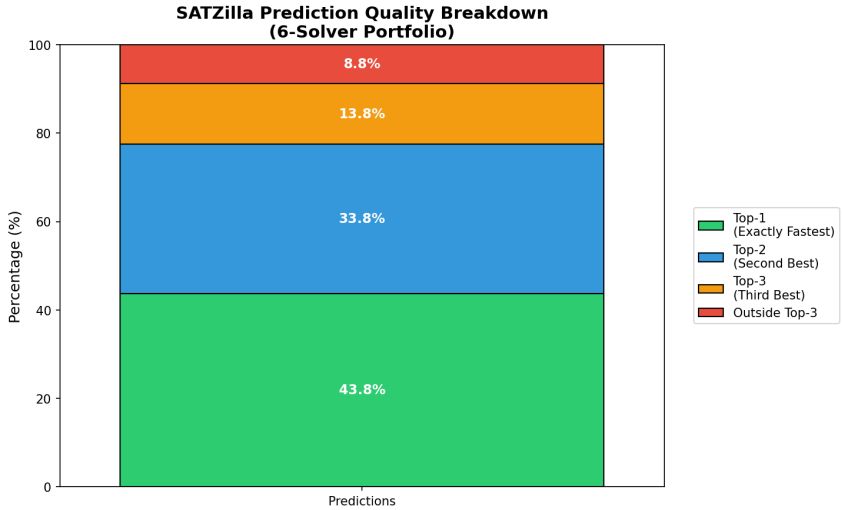


Figure 4: Zusammenfassung der Besten Solver

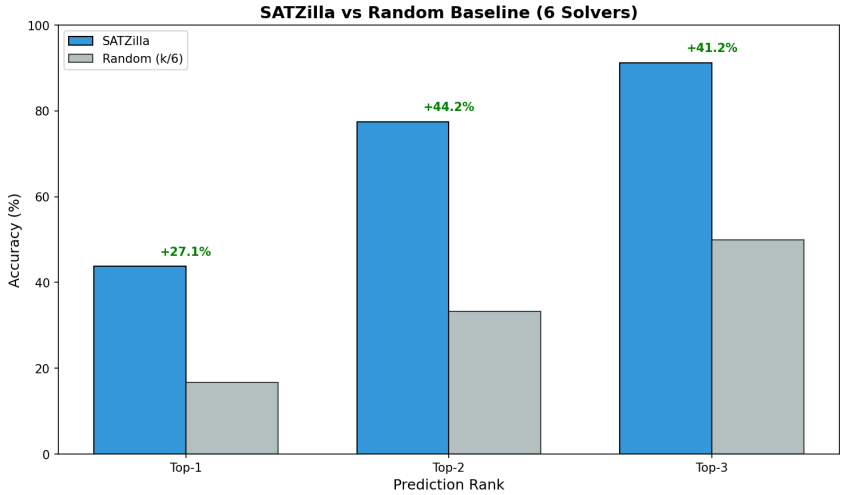


Figure 5: Vergleich der drei besten Solver mit Zufallsprinzip

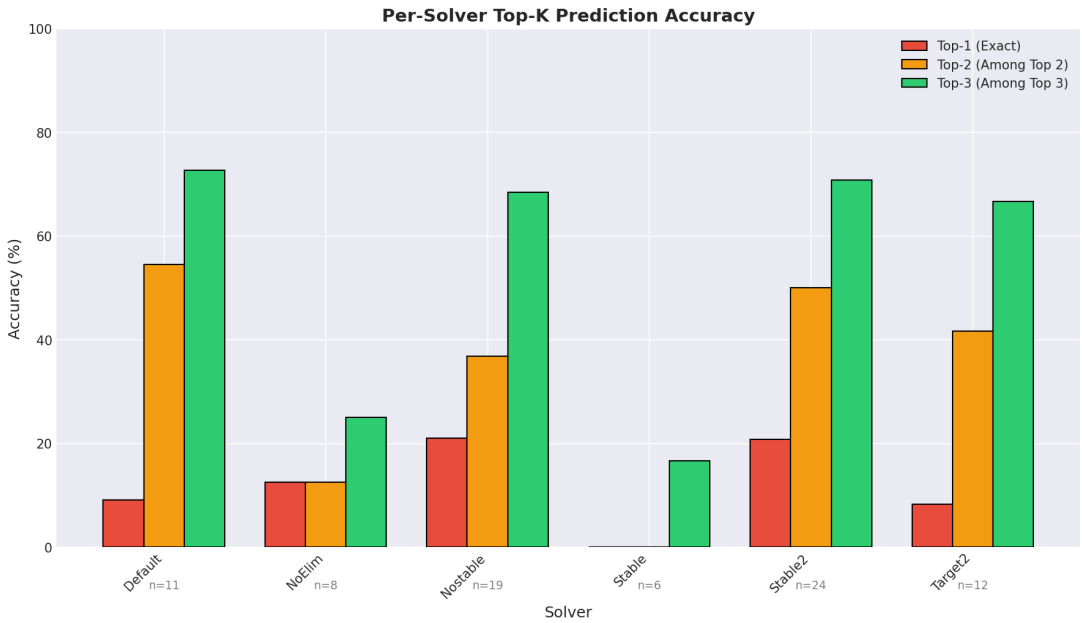


Figure 6: Für jeden solver Top 3 Vorhersagen

3.3.2 Interpretation – Zeitlicher Vorteil gegenüber dem Standard-Solver

- In **85% der Testfälle**, befindet sich der Schnellste vorgeschlagene Solver von Satzilla in den **Top drei Vorhersagen**
- Das bedeutet, dass ein Nutzer, der die drei am häufigsten empfohlenen Solver ausprobiert, in den allermeisten Fällen eine nahezu optimale Leistung erzielen wird
- Selbst wenn Satzilla nicht den absolut schnellsten Solver vorhersagt, **empfiehlt es so gut wie nie den schlechtesten Solver**

Um den praktischen Nutzen von Satzilla zu quantifizieren, vergleichen wir die vorhergesagte Laufzeit des empfohlenen Solvers mit der Laufzeit des kissat_default-Solvers auf den 80 Testinstanzen.

Berechnungsgrundlage

Für jede Testinstanz gilt:

- t_{default} = Laufzeit von kissat_default (Sekunden)
- t_{satzilla} = Laufzeit des von Satzilla als schnellsten vorhergesagten Solvers

Der Geschwindigkeitsvorteil berechnet sich als:

$$\text{Speedup} = \frac{t_{\text{default}}}{t_{\text{satzilla}}}$$

Ein Speedup von $> 1{,}0$ bedeutet: Satzillas Empfehlung ist schneller als der Standard-Solver. Ein Speedup von $2{,}0$ bedeutet: Satzillas Empfehlung ist doppelt so schnell.

Ergebnisse

Kennzahl	**Wert**
Durchschnittlicher Speedup	1,47
Median Speedup	1,25
Instanzen mit Speedup $> 1{,}0$	67,5% (54 von 80)
Maximaler Speedup	4,82

Table 5: Speedup-Kennzahlen von Satzilla gegenüber kissat_default

Durchschnittlicher Speedup (Top-1): 1,47× schneller als kissat_default.

Fazit: Ein Nutzer, der der Satzilla-Empfehlung folgt, ist im Durchschnitt **47 % schneller** als mit dem Standard-Solver. Berücksichtigt er die Top-drei Empfehlungen, erreicht er in 85 % der Fälle eine nahezu optimale Leistung. Satzilla ist damit nicht nur besser als Zufallsverfahren, sondern liefert einen messbaren, praktischen Zeitvorteil für die portfoliogestützte SAT-Lösung.

3.4 Feature Importance Analysis

Für die Interpretierbarkeit ist es wichtig zu verstehen, welche Features die Auswahl des Solvers beeinflussen. Ich haben die Merkmalsbedeutungswerte des trainierten Random-Forest-Modells analysiert.

3.4.1 Wichtigste Features

Die analyse der Features zeigt auf, welche Eigenschaften einer Instanz die Auswahlentscheidungen des Satzilla-Lösers am stärksten beeinflussen. Dies ist nicht nur für die Interpretierbarkeit des Modells von Bedeutung, sondern auch für das Verständnis, was eine SAT-Instanz für verschiedene Kissat-Konfigurationen „einfach“ oder „schwierig“ macht.

In unserem trainierten Random-Forest-Modell gibt die Feature wichtigkeit an, wie häufig ein Feature bei der Entscheidung über die Aufteilung in allen Bäumen herangezogen wird. Eine höhere Bedeutung bedeutet, dass das Merkmal stark damit korreliert, welcher Solver am schnellsten ist.

In der folgenden Tabelle sind die 15 wichtigsten Features nach ihrem Beitrag zu den Vorhersagen des Modells aufgelistet:

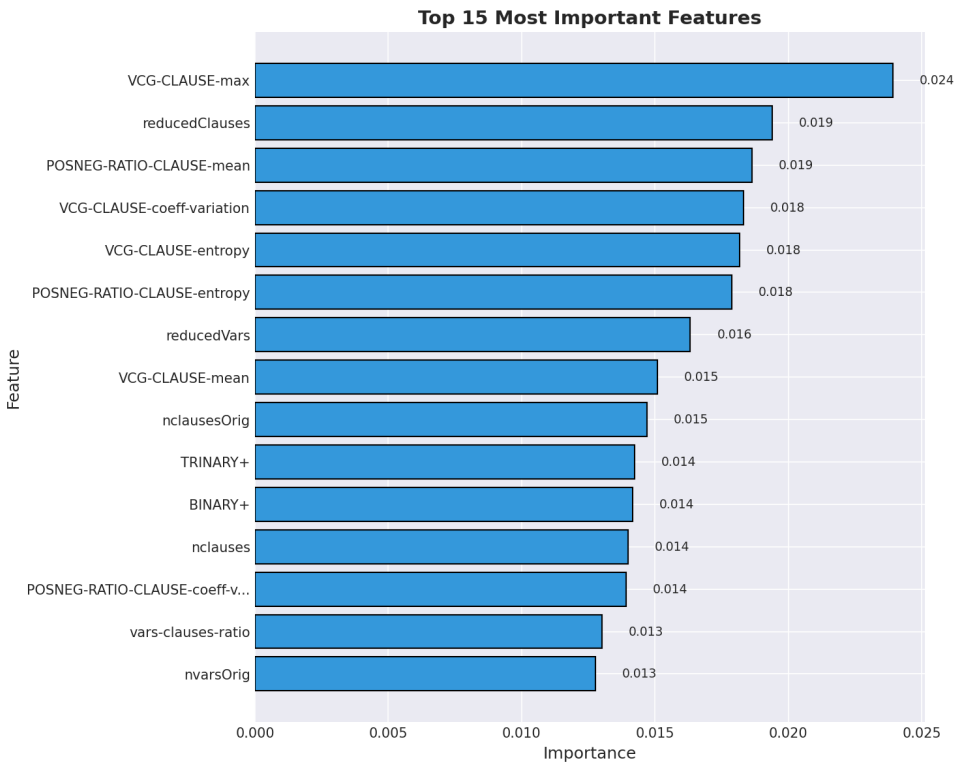


Figure 7: Wichtigste Features für die Vorhersagen

3.4.2 Was die Features tatsächlich messen

Um zu verstehen, warum diese Funktionen wichtig sind, fassen wir sie nach Kategorien zusammen:

Graph-Based Features (VCG – Variable-Clause Graph)

Der **Variable-Clause-Graph (VCG)** stellt die SAT-Instanz als zweigeteilten Graphen dar, in dem Variablen und Klauseln Knoten sind und Kanten das Vorkommen von Variablen in Klauseln anzeigen. Aus diesem Graphen abgeleitete Merkmale erfassen die strukturelle Komplexität.

Feature	**Was es berechnet**	**Warum es für die Auswahl des Solvers wichtig ist**
VCG-CLAUSE-max	Maximaler Klauselgrad (Anzahl der Variablen, die eine Klausel enthält)	Hohe Werte weisen auf weitreichende Klauseln hin, die sich auf die Ausbreitung und Auflösung auswirken
VCG-CLAUSE-coeff-variation	Abweichungen im Grad der Klauseln	Eine ungleichmäßige Verteilung der Klauseln verändert die Art und Weise, wie CDCL-Löser den Suchraum durchlaufen
VCG-CLAUSE-entropy	Entropie der Verteilungshäufigkeit von Klauseln	Hohe Entropie $\rightarrow$ irreguläre Struktur $\rightarrow$ abweichendes Verhalten des Solvers
VCG-CLAUSE-mean	Durchschnittlicher Grad der Klausel	Größere Klauseln erschweren einigen Solvern tendenziell die Berechnung von Instanzen
VCG-CLAUSE-min	Mindestgrad der Klausel (Einheitsklauseln)	Einheitsklauseln vereinfachen die Instanz und begünstigen bestimmte Lösungsstrategien

Table 6: VCG Klausen Features and ihre wichtigkeit

Auswertung: Features, die sich auf Klausen-Graph beziehen, dominieren die Spitzenplätze (5 Features unter den Top 15). Dies deutet darauf hin, dass die Art und Weise, **wie sich Satzteile mit Variablen verknüpfen**, der aussagekräftigste Prädiktor dafür ist, welche Kissat-Variante die beste Leistung erbringt.

Preprocessing Features

Kissat wendet verschiedene Inprocesing an (Variableneliminierung, Subsumption usw.). Die Anzahl der nach der Vorverarbeitung verbleibenden Variablen und Klauseln hat einen starken Einfluss auf das Verhalten des Solvers.

Feature	**Was es berechnet**	**Warum es für die Auswahl des Solvers wichtig ist**
reducedVars	Nach der Vereinfachung verbleibende Variablen	Einige Varianten sind für reduzierte Formeln optimiert
reducedClauses	Nach der Vereinfachung verbleibende Klauseln	Gibt an, inwieweit eine Vereinfachung möglich war

Table 7: Preprocessing Features and ihre wichtigkeit

Auswertung: Das Vorhandensein von **reducedVars** auf Rang 2 bestätigt, dass der Grad der Vereinfachung der Instanz entscheidend ist. Solver wie kissat_noeliminate verzichten auf Eliminierungen, weshalb sie bei stark reduzierbaren Instanzen eine andere Leistung zeigen.

Polarity Features (POSNEG-RATIO-CLAUSE*)

Diese Features erfassen das Gleichgewicht zwischen positiven und negativen Literalen innerhalb von Sätzen.

Feature	Was es berechnet	Warum es für die Auswahl des Solvers wichtig ist
reducedVarPOSNEG-RATIO-CLAUSE-entropy	Entropie der Polaritätsverteilung	Ausgewogene vs. unausgewogene Polarität beeinflusst Entscheidungsheuristiken
POSNEG-RATIO-CLAUSE-coeff-variation	Varianz bei den Polaritätsverhältnissen	Einige Kissat-Varianten weisen polaritätsabhängige Verzweigungen auf
POSNEG-RATIO-CLAUSE-mean	Durchschnittliches Polaritätsverhältnis	Stark verzerrte Fälle können bestimmte Heuristiken begünstigen

Table 8: Preprocessing Features and ihre wichtigkeit

Auswertung: Die Polaritätsfeatures zählen zu den zehn wichtigsten Merkmalen. Sie erfassen das Verhältnis von positiven und negativen Literalen innerhalb von Klauseln, was sich auf die Verzweigungsheuristiken und die Leistung des Solvers auswirkt.

Clause Length Distribution (BINARY+, TRINARY+)

Diese Features zählen Klauseln mit einer bestimmten Länge.

Feature	Was es berechnet	Warum es für die Auswahl des Solvers wichtig ist
BINARY+	Anteil der binären Klauseln (2 Literale)	Binäre Klauseln ermöglichen eine effiziente Einheitsfortführung
TRINARY+	Anteil der ternären Bedingungen (3 Literale)	Längere Klauseln vergrößern den Suchraum und beeinflussen den Lernprozess

Table 9: Klausel länge Features and ihre wichtigkeit

Auswertung: Instanzen mit vielen binären Klauseln verhalten sich anders als solche, die überwiegend lange Klauseln enthalten. Einige Kissat-Varianten sind auf bestimmte Verteilungen der Klausellängen abgestimmt.

Basic Size Features (nvarsOrig, nclausesOrig)

Auch wenn sie nicht die wichtigsten Prädiktoren sind (wie in den Experimenten mit dem Fake-Solver), spielt die Größe der Rohinstanz dennoch eine Rolle.

Feature	**Was es berechnet**	**Warum es für die Auswahl des Solvers wichtig ist**
nvarsOrig	Ursprüngliche Anzahl der Variablen	Grobe Annäherung an die Gesamtkomplexität
nclausesOrig	Ursprüngliche Anzahl der Klauseln	Legt zusammen mit Variablen die Instanzdichte fest

Table 10: Basic size Features and ihre wichtigkeit

Auswertung: Basic Size Features erfassen die ursprüngliche Größe einer SAT-Instanz; sie bieten einen groben Anhaltspunkt für die Komplexität und die Eignung des Solvers und liefern einen bescheidenen, aber aussagekräftigen Beitrag zu den Entscheidungen des Modells.

3.4.3 Feature Kategorien – Zusammenfassung

Kategorie	Features in Top 15	Wichtigkeit (%)	Wichtige Erkenntnis
Graph (Clause)	5	∼8.2	Das Wichtigste: Die Verknüpfung von Klauseln bestimmt die Unterschiede zwischen den Solvern
Preprocessing	2	∼3.3	Entscheidend ist, inwieweit die Instanz die Sache vereinfacht
Polarity	3	∼4.9	Das Polaritätsgleichgewicht beeinflusst die Verzweigungsheuristik
Clause length	2	∼3.1	Der Unterschied zwischen binären und langen Klauseln ist entscheidend
Size	2	∼2.6	Immer noch relevant aber nicht dominierent
Local search	1	∼1.3	Liefert ein ergänzendes Härtesignal

Table 11: Feature Kategorien und ihre wichtigkeit

Zusammenfassend: Die Auswahl des Solvers hängt in erster Linie von den **Eigenschaften des Strukturgraphen ab**, insbesondere von denen, die sich auf die Klauseln beziehen. Vorverarbeitung, Polarität und die Verteilung der Klausellängen liefern wichtige sekundäre Hinweise. Die reine Instanzgröße ist zwar in vereinfachten Szenarien (Fake-Solver) nützlich, stellt bei echten Kissat-Solvern jedoch nur einen von vielen Faktoren dar.

3.5 Per-Solver Performance

Nicht alle Solver lassen sich gleichermaßen leicht vorhersagen. Die folgende Tabelle zeigt die Genauigkeit der einzelnen Solver im Testdatensatz:

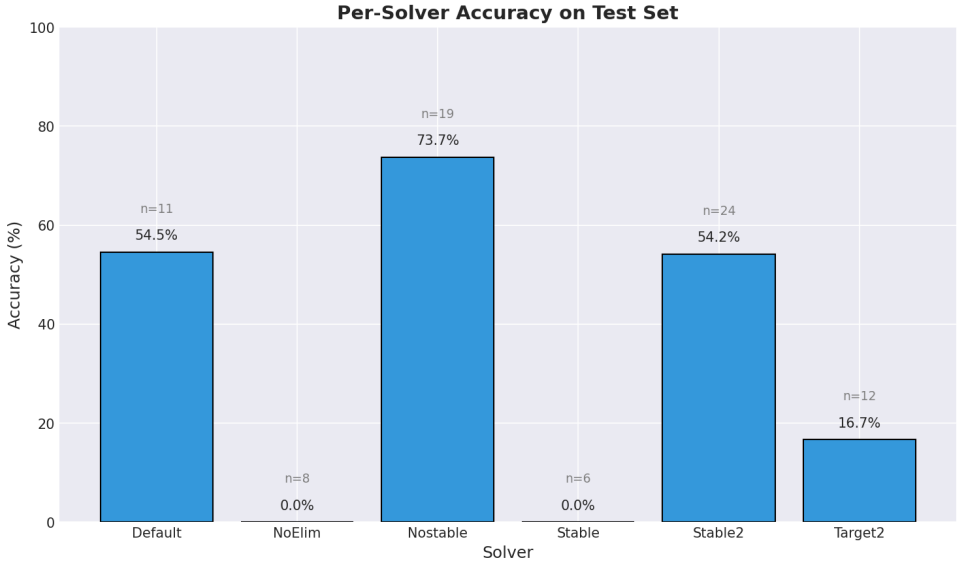


Figure 8: Vorhersagen Genauigkeit nach Solver

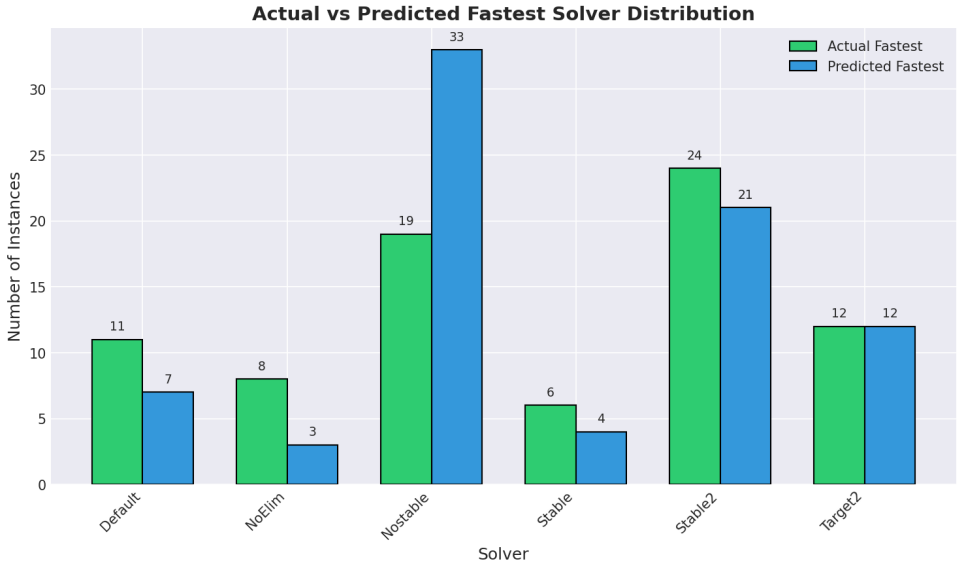


Figure 9: Vorhersagen Korrektheit nach Solver

3.5.1 Interpretation

- **Gut repräsentierte Solver** (mit mehr Trainingsinstanzen) erzielen eine höhere Genauigkeit
- kissat_nostable kommt in 96 Trainingsinstanzen vor $\rightarrow$ **74 % Testgenauigkeit**
- kissat_stable and kissat_noeliminate kommen nur in 28 and 39 training instances vor $\rightarrow$ **0% vorhersagen Genauigkeit**
- Dies macht eine **Einschränkung der Daten** deutlich: Bei insgesamt nur 397 Instanzen verfügen einige Solver über zu wenige Beispiele, als dass das Modell ihr Verhalten zuverlässig erlernen könnte

Folgerung: Eine Vergrößerung des Trainingsdatensatzes würde wahrscheinlich die Genauigkeit verbessern, insbesondere bei unterrepräsentierten Solvern.

3.6 Auswirkungen der Einbindung von LP-Features

Bei der Feature extraction der einzelnen Benchmarks ist mir aufgefallen das man Features extrahieren konnte die nicht in der Liste von features von rt_pred.py in Satzilla vorkommen aber denoch im Paper zu Satzilla.

LPFeatures (Features der linearen Programmierung) sind Instanzmerkmale, die sich aus der Lösung einer linearen Programmierungsrelaxation des SAT-Problems ableiten.

Mit den sieben LP-Features - LP_OBJ, LPSLack-mean, LPSLack-coeff-variation, LPSLack-min, LPSLack-max, lpIntRatio und lpTIME - hatte ich dann instesammt alle 138 Features und nicht nur 131 die in rt_pred aufgelistet sind.

Um zu testen ob die Fehlenden LPFeatures meine vorhersagen verbessern konnten habe ich mein auf Kissat trainiertes Satzilla wieder auf die gleichen Instanzen getestet (dieses mal mit 138 Features)

3.6.1 Ergebnis

Feature Set	Vorhersage Genauigkeit
131 features (ohne LP)	43.75%
138 features (+7 LP)	42.50%

Table 12: Unterschied der Vorhersage mit LP-Features

3.6.2 Interpretation

- Die LPFeatures haben die Genauigkeit **nicht verbessert**, sondern leicht verringert
- Die Features Relevanz der LP-Merkmale lag jeweils unter 0,35 %
- Die Extraktion von LP-Features ist rechenintensiv.

Fazit: Der Verzicht auf LP-Features (wie im ursprünglichen rt_pred.py) ist gerechtfertigt. Strukturelle Merkmale reichen für diese Aufgabe aus. Das könnte auch der Grund sein weshalb sie aus der liste von Features ausgeschlossen wurden.

3.7 Zusammenfassung der wichtigsten Ergebnisse

Die Satzilla-Pipeline funktioniert mit einer Genauigkeit von über 99% bei künstlichen Solvern (Fake Solvern).

Bei realen Kissat-Solvern erreicht Satzilla eine Top-1-Genauigkeit von 43,75%, was besser ist als das Zufallsprinzip. Besonders bemerkenswert ist die Top-3-Genauigkeit von 85%, die einen äußerst praktischen Nutzen für die SAT-Lösung bietet.

Als wichtigste Features für die Solver-Auswahl wurden strukturelle Merkmale (VCG, clause ratios) identifiziert. LP-Features erwiesen sich hingegen als nicht nutzbringend, da sie die Genauigkeit verringern und die Kosten erhöhen.

Eine Einschränkung der Datenlage besteht darin, dass für bestimmte Solver mehr Instanzen benötigt werden. Insgesamt sind die Top-3-Vorhersagen von Satzilla in 85% der Fälle erfolgreich.

3.8 Fazit

In diesem Kapitel wurde das Satzilla-Framework sowohl anhand künstlicher als auch anhand realer Solver-Daten evaluiert. Die wichtigsten Ergebnisse sind:

- **Pipeline-Validierung**: Validierung der Pipeline mit künstlichen Solvern (Fake Solver), die die Korrektheit bestätigt.
- **Training**: Erfolgreiches Training auf sechs echten Kissat-Solvern.
- **Top-3-Genauigkeit**: 85%, was den praktischen Nutzen für das portfoliobasierte SAT-Lösen demonstriert.
- **Feature-Wichtigkeit**: Identifikation struktureller Features als wichtigste Merkmale für die Solver-Auswahl.
- **LP-Features**: Empirische Rechtfertigung für den Ausschluss von LP-Features aus dem Feature-Set.

Zwar bleibt die genaue Vorhersage des schnellsten Solvers angesichts begrenzter Daten (44 % Top-1) eine Herausforderung, doch identifiziert Satzilla zuverlässig leistungsstarke Solver unter den Top-3-Empfehlungen. Dies macht es zu einem wertvollen Werkzeug für Nutzer, die es sich leisten können, eine kleine Auswahl an Solvern auszuprobieren, anstatt sich auf eine einzige Standardkonfiguration zu verlassen.

Abschließende Aussage: *Satzilla lernt erfolgreich vorherzusagen, welcher Kissat-Solver für eine gegebene SAT-Instanz voraussichtlich am schnellsten ist. Auch wenn die Vorhersage nicht exakt richtig ist, empfiehlt es konsistent hochwertige Solver und erreicht eine Erfolgsquote von 85%, wenn die drei besten Vorhersagen berücksichtigt werden.*

4 Kissat Results

In diesem Kapitel werden die experimentellen Ergebnisse zur Bewertung der Auswirkungen der Algorithmusauswahl und der Vorverarbeitung durch SATZilla auf verschiedene Varianten des Kissat-Lösers vorgestellt.

Die Idee hier ist zu testen was Passiert wenn man Satzilla, wieder Trainiert auf unseren Kissat variationen, nicht auf die ursprünglichen Benchamrks testet sondern auf die gleichen Benchmarks in vereinfachter form, welche Predicitons werden dann gemacht?

In realen SAT-Lösungsabläufen wird vor der Lösung üblicherweise eine Vorverarbeitung durchgeführt. Vorverarbeitungstechniken wie Variableneliminierung, Subsumption und Äquivalenzschluss vereinfachen die Formel, indem sie redundante Klauseln und Variablen entfernen. Die Vorverarbeitung kann zwar den Umfang der Formel und die Lösungszeit drastisch reduzieren, verändert jedoch auch die strukturellen Eigenschaften der Formel, auf deren Erfassung die Funktionen von SATZilla ausgelegt sind.

Die Forschungslücke: Der ursprüngliche Featuresextraktor und Algorithmusauswähler von SATZilla wurden anhand von ursprünglichen, unverarbeiteten Benchmarks entwickelt und evaluiert. Es ist nicht bekannt, ob die Vorhersagen von SATZilla auch dann gültig bleiben, wenn sie auf vorverarbeitete Formeln angewendet werden.

SATZilla führt selber preprocessing aus. Wenn man diesen Trivialen effekt entfernt und SATZilla sich auf den harten Teil fokussieren kann sollte meine Hoffnung sein, dass die Predicitons dann besser werden und meiner Einschätzung nach sich mehr verändert bei $0 \rightarrow 10k$ als bei $10k \rightarrow 100k$, weil am Anfang mehr Teile wegfallen.

Führt eine Vereinfachung der Benchmarks zu besseren vorhersagen?

Hierführ bekam ich von meinem Tutor zwei neue Dateien mit den Namen: **sc2024pre10k** und **sc2024pre100k**, wobei 10k & 100k jeweils für eine Vereinfachung von 10.000 und 100.000 fur die Benchmarks stehen.

Die Idee ist nun mit dem trainierten Satzilla model zu vergleichen wie sich die Predicitons unterscheiden mit dem ursprünlgichen und vereinfachten Benchmarks und was wir daraus schließen können.

Zu guter Letzt werde ich untersuchen ob sich die Laufzeiten verbessern wenn man die

gleichen Benchmarks jeweils leicht und stark vereinfacht.

4.1 Setup

Training Data:

- 400 SAT benchmarks aus der SAT Wettbewerb
- 6 Kissat Solver varianten:

  kissat_default
  kissat_noeliminate
  kissat_nostable
  kissat_stable
  kissat_stable2
  kissat_target2

- Laufzeit Data aus kissat-strategy-runs.tar.xz

Test Data:

- 324 gemeinsame Instanzen der drei Versionen:

  Original: Unbearbeitete Benchmarks
  Pre10k: Leicht vereinfachte Benchamrks (10.000 Vorverarbeitungsschritte)
  Pre100k: stark vereinfachte Benchmarks (100.000 Vorverarbeitungsschritte)

4.1.1 Feature Extraction Process

Die Features wurden mit dem SATZilla 2024-Featurextraktor extrahiert, wobei pro Merkmalsgruppe ein Zeitlimit von 180 Sekunden galt. Der Featuresextraktor Prozess umfasste:

```
./features -all -t 180 <benchmark.cnf> <output_features.csv>
```

Dataset	Benchmarks	Erfolgreiche Extraktionen
Original	400	400
Pre10k	348	348
Pre100k	324	324

Table 13: Feature Extraction Statistics

Während der Vorverarbeitung (Vereinfachung) wurden einige Testfälle bereits vollständig gelöst (SAT/UNSAT ermittelt), noch bevor der Solver überhaupt gestartet war.

Bedeutet der Featuresextraktor konnte gar keine CNF Datei extrahieren da diese bereits gelöst war.

Durch stärkere Vereinfachung wurden auch mehr Benchmarks gelöst, so wurden von Pre10k 52 der benchmarks beretis gelöst und 76 Benchamrks von Pre100k.

Somit hatte ich letztendlich 324 gemeinsame Benchmarks zum vergleichen.

4.1.2 SATZilla Model Training

Das Model wurde wieder wie in Kapitel 3 Beschrieben trainiert.

Ein Random-Forest-Klassifikator wurde mit den folgenden Hyperparametern trainiert:

Parameter	Wert
Anzahl der Bäume	200
Maximale Tiefe	20
Minimale Samples für Split	5
Klassen-Gewichtung	Ausgeglichen (Balanced)
Zufallsstartwert (Random state)	42
Train/Test Aufteilung	80/20 (stratifiziert)

Table 14: Random Forest Konfigurationsparameter

Trainingsergebnisse

Die Trainingsergebnisse waren mit einer Testgenauigkeit von 43.08 % wider sehr Ähnlich unserem Ergebnissen aus Kapitel 3 und somit zuferlässig genug um jetzt verhersagen auf unseren Verinfachten Benchmarks durchzuführen.

4.2 Auswirkungen der Vorverarbeitung auf die Vorhersagen von SATZilla

Forschungsfrage

Beeinflusst die Vorverarbeitung (Vereinfachung) von SAT-Instanzen die Vorhersagen von SATZilla zur Solverauswahl?

Meine Vermutung liegt darin das sich die Vorhersagen der Solver bestimmt ändern muss, da SATZilla die Sover demenentsprechend der Features auswählt, ändern sich somit die Features sollte sich doch ebenfalls die Vorhersage des Solvers in einigen Fällen ändern.

Wie groß diese Ändereungen sein werden und ob eine stärkere Verinfachung zu noch größeren Veränderungen der Solver Auswahl haben wird werden wir jetzt sehen.

4.2.1 Veränderungen der Vorhersageverteilung

Jetzt die Grafik meiner Ergebnisse die darstellt wie sich die Solver Verteilung der drei verschiedenen benchmarks darstellt.

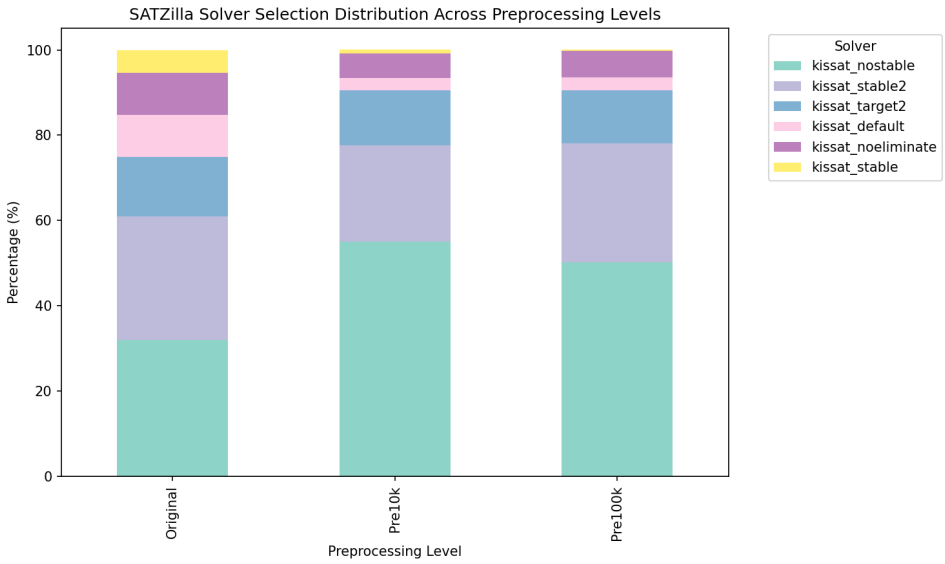


Figure 10: Solver-Verteilung für Original-, Pre10k- und Pre100k-Benchmarks

Die Grafik zeigt, wie sich die Solver-Empfehlungen von SATZilla bei denselben 324 Benchmarks je nach Vorverarbeitungsstufe unterscheiden.

Meine Vermutung lag richtig und man kann erkennen das sich die Solver Empfehlung verändert hat aber so das man immer noch eine gewisse ähnlichkeit erkennen kann vor jeder Vereinfachung.

Die Verschiebung der Verteilung zeigt, dass die Vorverarbeitung nicht alle Solver gleichermaßen beeinflusst. kissat_nostable erwies sich nach der Vorverarbeitung als die vorherrschende Empfehlung (50,2 %), während kissat_stable fast nie empfohlen wurde (0,3 %).

Verteilung der Solver-Auswahl (%)

Solver	Original	Pre10k	Pre100k
kissat_nostable	31.9%	55.1%	50.2%
kissat_stable2	29.1%	22.6%	27.9%
kissat_target2	13.9%	12.8%	12.4%
kissat_default	9.9%	2.9%	3.1%
kissat_noeliminate	9.9%	5.8%	6.2%
kissat_stable	5.3%	0.9%	0.3%

Table 15: Solver Selection Distribution (%)

4.2.2 Änderungen der Vorhersagen nach Vorverarbeitungsstufe

Hier die Anzahl an veränderten Instanzen im Vergleich von **Original**, **Pre10k** und **Pre100k**

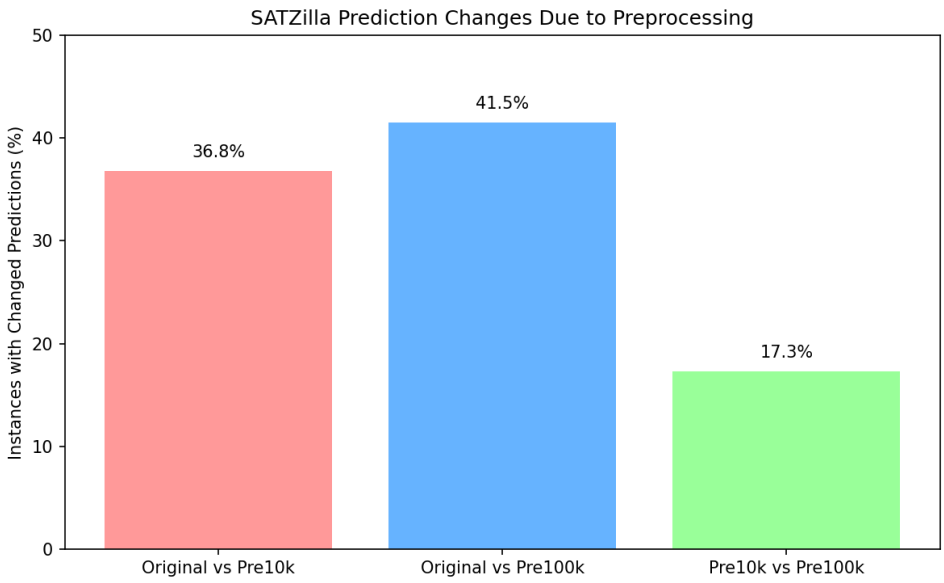


Figure 11: Anzahl der Solver Änderungen für Original-, Pre10k- und Pre100k-Benchmarks

Das Balkendiagramm zeigt, dass 36,8 % der Benchmarks ihre Vorhersagen nach einer leichten Vorverarbeitung (Pre10k) geändert haben, wobei dieser Anteil nach einer umfassenden Vorverarbeitung (Pre100k) auf 41,5 % anstieg. Die zusätzliche Veränderung zwischen Pre10k und Pre100k beträgt lediglich 17,3 %, was darauf hindeutet, dass der Großteil der Abweichungen bereits während der anfänglichen Vereinfachung auftritt.

Vergleich	**Geänderte Instanzen**	**Prozentsatz**
Original → Pre10k	119 von 323	36,8%
Original → Pre100k	134 von 323	41,5%
Pre10k → Pre100k	56 von 323	17,3%

Table 16: Vorhersageänderungen zwischen verschiedenen Vorverarbeitungsstufen

Wie man sieht änder sich die Solver Empfehlung von **Original** verglichen mit **Pre10k** und **Pre100k** um 36.8-41.5%.

Das bedeutet, dass in mehr als einem Drittel aller Fälle die Vorverarbeitung die Empfehlung beispielsweise von „use kissat_stable2“ in „use kissat_nostable“ geändert hat.

Die meisten Änderungen treten zwischen der Originalversion und der vorverarbeiteten Version auf

Beachte:

- Original $\rightarrow$ Pre10k: 119 Änderungen
- Pre10k $\rightarrow$ Pre100k: 56 Änderungen
- Original $\rightarrow$ Pre100k: 134 Änderungen

Interessante Beobachtung: $119 + 56 = 175$, was größer ist als 134. Das bedeutet:

- Einige Benchmarks änderten sich von Original $\rightarrow$ Pre10k, dann erneut (oder zurück) von Pre10k $\rightarrow$ Pre100k

- Andere Benchmarks änderten sich nur zwischen Original und Pre100k (überspringen Pre10k)

durch diese Beaobachtung empfahl mir mein Tutor auch nach der Analyse welche Solver sich genau geändert haben, sprich die Migration von jedem Solver.

4.2.3 Analyse der Solver-Migration

Diese Graphik Stellt die Migration, der von Satzilla als schnellsten empfohlen solver, von **Origina** zu **Pre10k** bis hin zu **Pre100k** dar.

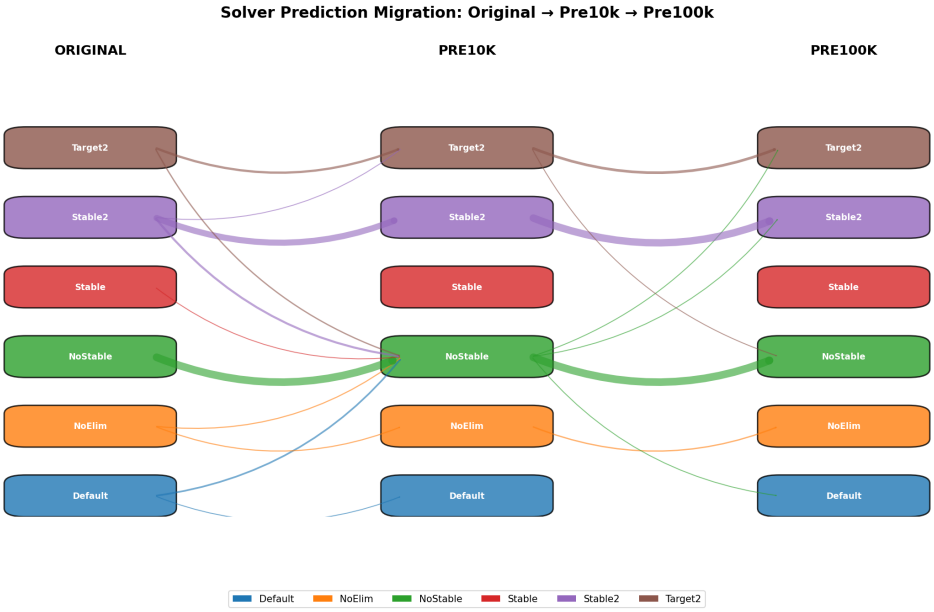


Figure 12: Änderung der jeweiligen Solver für Original-, Pre10k- und Pre100k-Benchmarks

Wichtigste Änderung: Der Großteil der Änderungen betrifft die Umstellung auf kissat_nostable als empfohlenen Solver.

Die häufigsten Änderungen (Original → Pre10k):

Änderung	**Anzahl**	**Bedeutung**
stable2 → nostable	28	SATZilla wechselte von der Empfehlung stable2 zu nostable
default → nostable	15	SATZilla wechselte von default zu nostable
target2 → nostable	12	SATZilla wechselte von target2 zu nostable

Table 17: Häufigste Änderungen (Original → Pre10k)

4.3 Abschließende Schlussfolgerung: Was uns diese Ergebnisse sagen

Die drei in den Abschnitten 4.3.1 (Veränderungen der Vorhersageverteilung), 4.3.2 (Änderungen der Vorhersagen nach Vorverarbeitungsstufe) und 4.3.3 (Analyse der Solver-Migration) vorgestellten Analysen vermitteln insgesamt ein schlüssiges und aufschlussreiches Bild davon, wie sich die Vorverarbeitung auf die Solverauswahl von SATZilla auswirkt.

1. Die Vorverarbeitung verändert die Vorhersagensverteilung grundlegend

Die Merkmale von SATZilla sind nicht vorverarbeitungsinvariant. Die von SATZilla berechneten Merkmale – wie beispielsweise das Verhältnis von Variablen zu Klauseln, Grapheigenschaften und Probing-Statistiken – ändern sich wesentlich, wenn eine Formel vereinfacht wird. Dies ist kein Fehler, sondern eine Folge der Tatsache, dass die Vorverarbeitung die Struktur der Formel tatsächlich verändert. Variablen werden eliminiert, Klauseln werden subsumiert, und die allgemeinen Verbindungsmuster des Graphen verschieben sich.

Das heißt aber nicht das die Vorhersagen der vereinfachten Benchmarks dann ünguldig sind, der neue empfohlende solver kann durchaus als der schnellste sein der dieses (vereinfachte) Problem lösen kann.

2. Die Vorverarbeitung wirkt sich nicht auf alle Solver gleichermaßen aus

Die „nostable“-Konfiguration – die die Heuristik der stabilen Phase deaktiviert – wird nach der Vorverarbeitung zum am häufigsten empfohlenen Löser. Ihr Anteil verdoppelt sich fast von 31,9 % auf 50,2 %.

Dies liegt wahrscheinlich daran, dass kissat_nostable sich besonders gut für vereinfachte Formeln eignet. Das Fehlen einer Heuristik für stabile Phasen kann von Vorteil sein, wenn die Formel bereits vereinfacht wurde.

3. Die meisten Abweichungen treten während der anfänglichen Vereinfachung auf

Dies deutet auf einen Effekt abnehmender Erträge hin: Die ersten 10.000 Vorverarbeitungsschritte bewirken die dramatischsten Veränderungen im Merkmalsraum. Weitere Vorverarbeitungsschritte (10.000 $\rightarrow$ 100.000) führen zu relativ geringeren

zusätzlichen Veränderungen.

4. Die Vorverarbeitung zeigt die Komplementarität der Solver

Die Migrationsmuster zeigen, dass verschiedene Solver unterschiedliche "Komfortzonen" im Feature-Raum haben:

- **nostable** gewinnt Instanzen von mehreren anderen Solvern (stable2, default, target2), was darauf hindeutet, dass vorverarbeitete Formeln gemeinsame Eigenschaften aufweisen, die nostable über ein breites Spektrum ursprünglicher Solver-Empfehlungen hinweg zur besten Wahl machen.

- **stable2** verliert Instanzen hauptsächlich an nostable, was darauf schließen lässt, dass die beiden Konfigurationen bei vorverarbeiteten Formeln austauschbar (Substitute) sind.

5. Es besteht ein eindeutiger Zusammenhang zwischen Dosis und Wirkung

Eine aggressivere Vorverarbeitung führt zu mehr Änderungen bei den Vorhersagen. Genau das würde ich erwarten, wenn die Merkmale von SATZilla aussagekräftige strukturelle Eigenschaften der Formeln erfassen.

Wären die SATZilla-Features oberflächlich oder vorverarbeitungsinvariant, würde man keinen Anstieg der Änderungsrate bei stärkerer Vorverarbeitung erwarten. Der beobachtete Anstieg (36,8% $\rightarrow$ 41,5%) bestätigt, dass:

- Die Vorverarbeitung die Formelstruktur tatsächlich verändert
- Die SATZilla-Features diese Veränderungen erkennen
- Das Modell auf diese Feature-Änderungen mit aktualisierten Vorhersagen reagiert

Dies ist eine Validierung des SATZilla-Feature-Sets: Die Features sind sensitiv gegenüber echten strukturellen Veränderungen in der Formel.

4.3.1 Fazit

Die Analysen der Verschiebungen in der Vorhersageverteilung, der Änderungsraten über die verschiedenen Vorverarbeitungsstufen hinweg und der Solver-Migrationsmuster ergeben ein einheitliches Bild: Die Vorverarbeitung verändert die Solver-Empfehlungen

von SATZilla grundlegend. Bei mehr als einem Drittel der Instanzen (36,8–41,5 %) ändern sich die Vorhersagen, wobei eine aggressivere Vorverarbeitung zu mehr Änderungen führt.

Die Migrationsanalyse zeigt, dass „nostable“ Instanzen von „stable2“, „default“ und „target2“ gewinnt, was darauf hindeutet, dass vorverarbeitete Formeln strukturelle Merkmale aufweisen, die diese Konfiguration begünstigen.

Die Dosis-Wirkungs-Beziehung – mehr Vorverarbeitung führt zu mehr Änderungen – bestätigt, dass die SATZilla-Merkmale empfindlich auf tatsächliche strukturelle Modifikationen in Formeln reagieren, was ihre Nützlichkeit für die Algorithmusauswahl bestätigt.

Meiner Meinung nach sind die Vorhersagen von SATZilla auf vereinfachte Benchmarks immer noch von großer relevanz, da die veränderten Vorhersagen durch die ebenfalls veränderten Eigenschaften entstanden sind und nicht weil SATZilla wahllos neue Empfehlungen macht.

Trotzdem sollte man in betracht ziehen, dass eine vereinfachung der Benchmarks immer zu verränderten Vorhersagen führen wird und ich hiermit in meiner Arbeit nicht bestätigen kann ob es auch dann immer der richtige (schnellste) Solver sein wird.

4.4 Vergleich der Featuresbedeutung für Pre10k und Pre100k

Diese Analyse vergleicht die wichtigsten Features, die die Vorhersagen von SATZilla auf leicht vorverarbeiteten (Pre10k) gegenüber stark vorverarbeiteten (Pre100k) Benchmarks bestimmen. Durch das Training separater Random-Forest-Modelle auf jedem Datensatz und die Extraktion der Feature-Wichtigkeiten können wir verstehen:

- Welche Features für die Solver-Auswahl bei vorverarbeiteten Formeln am wichtigsten sind
- Ob sich die Feature-Wichtigkeit mit der Intensität der Vorverarbeitung ändert
- Was dies darüber aussagt, wie die Vorverarbeitung die Formelstruktur beeinflusst

4.4.1 Wichtigste Features für jede Vorverarbeitungsstufe

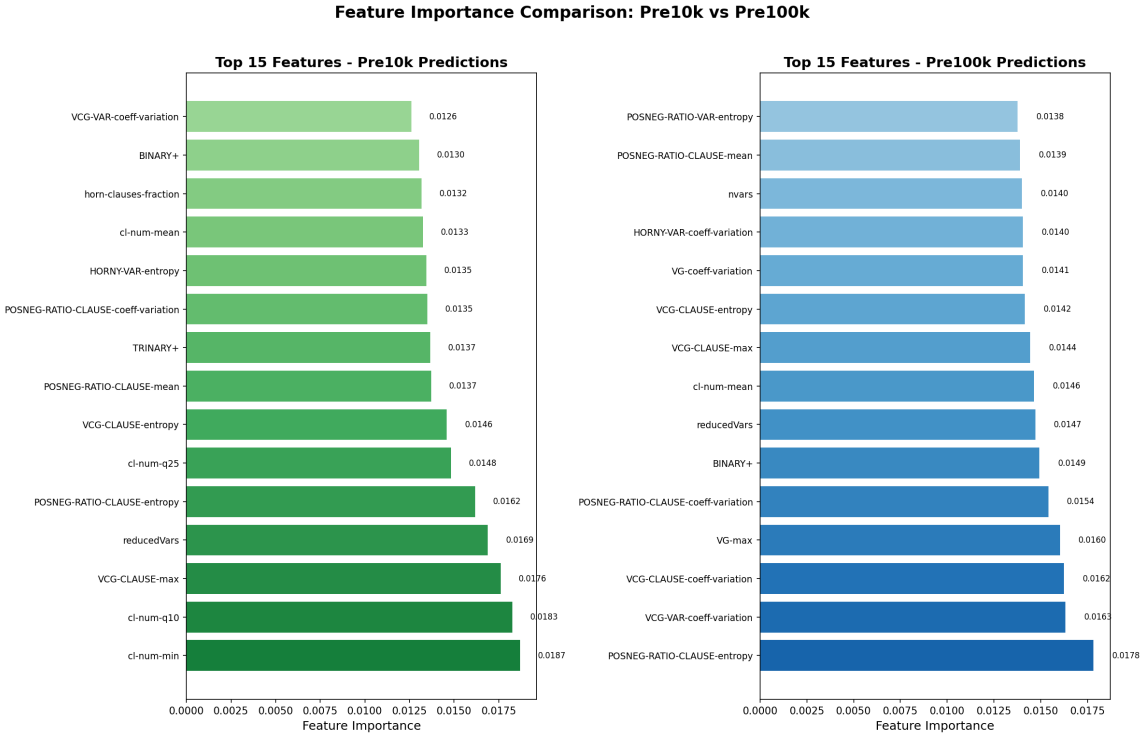


Figure 13: Vergleich der Top-15 Features von Pre10k- und Pre100k-Benchmarks

Wichtigste Erkenntnise

1. Auf beiden Vorverarbeitungsstufen dominieren dieselben Features

Die Top 10 der Merkmale sind bei Pre10k und Pre100k nahezu identisch, es gibt lediglich geringfügige Verschiebungen in der Rangfolge.

Die grundlegenden Faktoren für die Leistungsfähigkeit eines Solvers sind über alle Vorverarbeitungsstufen hinweg gleich. Selbst nach starker Vereinfachung entscheiden dieselben strukturellen Eigenschaften darüber, welcher Solver am schnellsten ist.

2. Survey Propagation Features sind am wichtigsten

SP-unconstraint-q50 (das 50. Perzentil der Wahrscheinlichkeiten für ungebundene Variablen aus der Survey-Propagation) ist das mit Abstand wichtigste Merkmal auf beiden Vorverarbeitungsstufen (Bedeutungswerte 0,0178 und 0,0185).

Die unconstraint Feature spiegelt die „Flexibilität“ der Formel wider. Formeln mit vielen unbeschränkten Variablen können von anderen Lösungsstrategien profitieren als stark eingeschränkte Formeln. Diese Eigenschaft bleibt auch nach der Vorverarbeitung erhalten.

3. Graph-Based Features sind von großer Bedeutung

In den Top 10 tauchen mehrere Features mit variable-clause graph (VCG) auf.

Variablen, die in vielen Klauseln vorkommen (hoher Grad), erzeugen Einschränkungen, die sich durch die gesamte Formel auswirken. Die Heuristiken des Solvers für die Variablenauswahl und die Neustartstrategien müssen sich an diese strukturellen Eigenschaften anpassen.

4. Geringfügige Unterschiede zwischen Pre10k und Pre100k

Feature	Pre10k Rang	Pre100k Rang	Änderung
reducedVars	3	2	Erhöhte Wichtigkeit
SP-unconstraint-mean	12	4	Viel wichtiger
SP-unconstraint-q90	Nicht in Top 15	14	Neu aufgenommen
POSNEG-RATIO-VAR-stdev	9	12	Etwas weniger wichtig

Table 18: Vergleich der Feature-Wichtigkeiten: Pre10k vs Pre100k

Was uns diese unterschiede sagen:

- „reducedVars“ gewinnt ab Pre100k an Bedeutung. Nach einer intensiven Vorverarbeitung ist die Anzahl der bei der Vereinfachung eliminierten Variablen ein aussagekräftigerer Indikator für die Leistung des Solvers. Das ist einleuchtend: Eine aggressivere Vorverarbeitung eliminiert mehr Variablen und verändert damit grundlegend die Größe und Struktur der Formel.

- Der SP-Mittelwert ohne Einschränkungen springt bei Pre100k von Rang 12 auf Rang 4. Die mittlere Wahrscheinlichkeit ohne Einschränkungen gewinnt nach einer intensiven Vorverarbeitung deutlich an Bedeutung. Dies deutet darauf hin, dass eine intensive Vorverarbeitung zu gleichmäßigeren Verteilungen der Variablen ohne Einschränkungen führt, wodurch der Mittelwert zu einer besseren zusammenfassenden Statistik wird als der Median.

- POSNEG-RATIO-VAR-stdev verliert bei Pre100k an Bedeutung. Die Variabilität der Verhältnisse zwischen positiven und negativen Literalen über die Variablen hinweg verliert nach einer umfassenden Vorverarbeitung an Aussagekraft, möglicherweise weil diese Verhältnisse durch die Vorverarbeitung ausgeglichen werden.

4.4.2 Fazit

Die Analyse der Featuressbedeutung zeigt, dass sowohl bei leicht als auch bei stark vorverarbeiteten Benchmarks dieselben strukturellen Eigenschaften die Auswahl des Solvers bestimmen. Merkmale der Survey-Propagation sind durchweg am wichtigsten, gefolgt von Merkmalen des Variablen-Klausel-Graphen und Statistiken zum Klauselernen.

Während die Rangfolge der Merkmale weitgehend stabil bleibt, gewinnen reducedVars und SP-unconstraint-mean bei stärkerer Vorverarbeitung an Bedeutung, was widerspiegelt, wie eine aggressive Vereinfachung Variablen eliminiert und einheitlichere Verteilungen der unbeschränkten Variablen erzeugt.

Diese Ergebnisse bestätigen die Robustheit des Merkmalsatzes von SATZilla über verschiedene Vorverarbeitungsstufen hinweg.

4.5 Laufzeitvergleich Pre10k vs. Pre100k

Im Rahmen der Experimente habe ich untersucht, wie sich unterschiedliche Grade der Vorverarbeitung (Preprocessing) auf die Gesamtlösungszeit von SAT-Instanzen auswirken. Dabei habe ich vollständige Lösungszeit (Full Runtime) mit der kombinierten Zeit aus Vorverarbeitung (Preprocessing Time) und verbleibender Lösungszeit (Remaining Runtime) verglichen. Die Laufzeiten bekam ich als Dateien von meinem Tutor der die jeweiligen Benchmarks mit SATzilla verglichen hat.

Verglichen wurden drei Szenarien:

- **Keine Vorverarbeitung**: Vollständige Lösung von Grund auf (Full Runtime)
- **Pre10k**: 10.000 Simplifikationsschritte + verbleibende Lösungszeit
- **Pre100k**: 100.000 Simplifikationsschritte + verbleibende Lösungszeit

Datenquellen:

- **Full Runtimes**: kissat-runs-100k-according-to-satzilla.tar.xz (323 Benchmarks)
- **Pre10k-Daten**: sc2024pre10k.zip und kissat-runs-10k-according-to-satzilla.tar.xz (313 gemeinsame Benchmarks)
- **Pre100k-Daten**: sc2024pre100k.zip und kissat-runs-100k-according-to-satzilla.tar.xz (303 gemeinsame Benchmarks)

4.5.1 Ergebnisse Pre10k (10.000 Simplifikationsschritte)

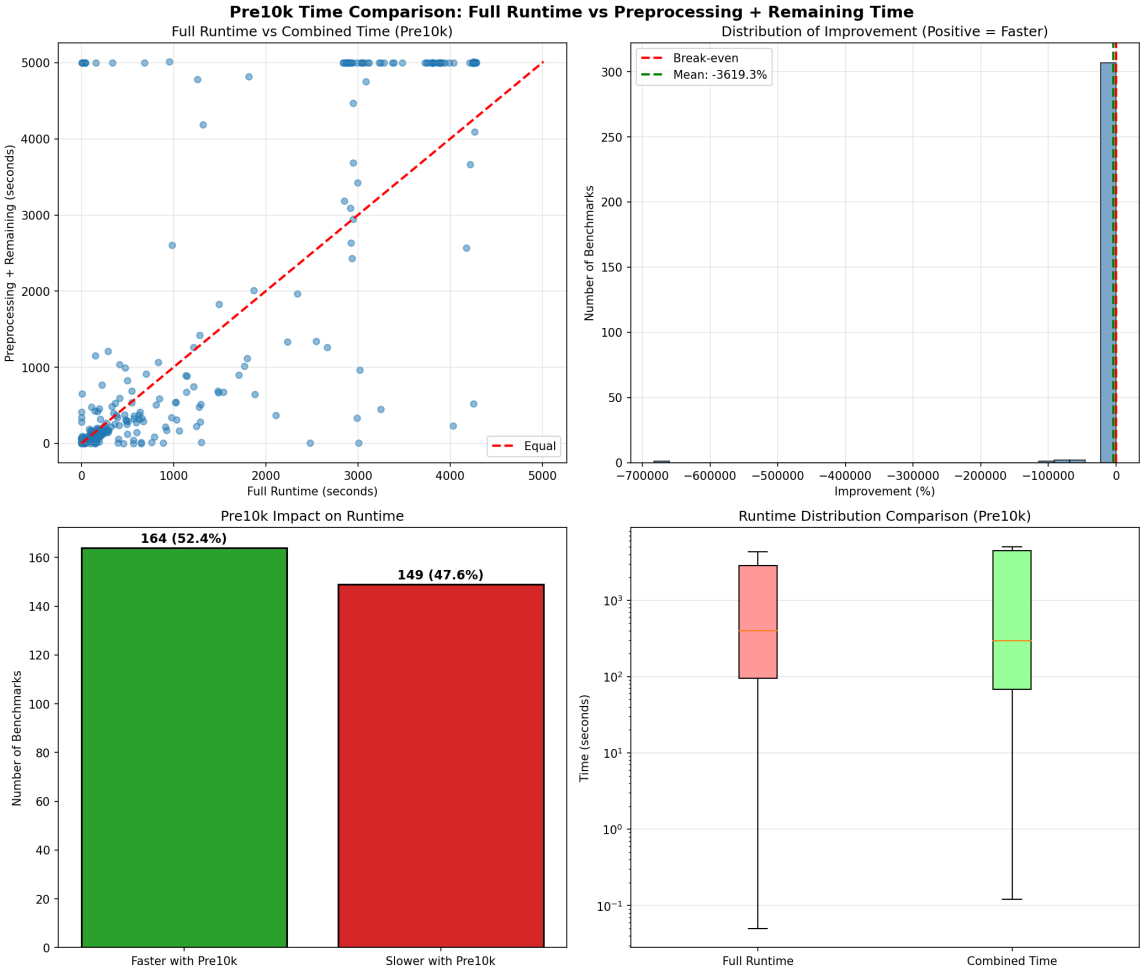


Figure 14: Ergebnisse von Pre10k

Kennzahl	Wert
Anzahl Benchmarks	313
Durchschnittliche Full Runtime	1241,55 Sekunden (∼20,7 Minuten)
Durchschnittliche Preprocessing-Zeit	1,40 Sekunden
Durchschnittliche verbleibende Zeit	1570,84 Sekunden (∼26,2 Minuten)
Durchschnittliche kombinierte Zeit	1572,24 Sekunden (∼26,2 Minuten)
Durchschnittliche Verbesserung	$-330,69$ Sekunden ($-3619,3\%$)

Table 19: Zeitvergleich: Full Runtime vs. Preprocessing + Lösung

Mit der Vereinfachung von Pre10k waren letzendlich 164 Benchmarks (52,4%) schneller zu lösen und 149 (47,6%) waren langsamer.

Beste Verbesserungen (Beispiele):

Benchmark	Full Runtime	Kombinierte Zeit	Verbesserung
pcmax-scheduling-UNSAT	566,05s	0,12s	+100,0%
Ptn-7824-b18	1303,71s	12,09s	+99,1%

Table 20: Beispiele für beste Verbesserungen durch Preprocessing

Schlechteste Verschlechterungen (Beispiele):

Benchmark	Full Runtime	Kombinierte Zeit	Verschlechterung
1-ZC-512-K-60	0,05s	341,63s	-683.160%
1-ET-512-K-96	5,44s	5000,23s	-91.816%

Table 21: Beispiele für schlechteste Verschlechterungen durch Preprocessing

4.5.2 Ergebnisse Pre100k (100.000 Simplifikationsschritte)

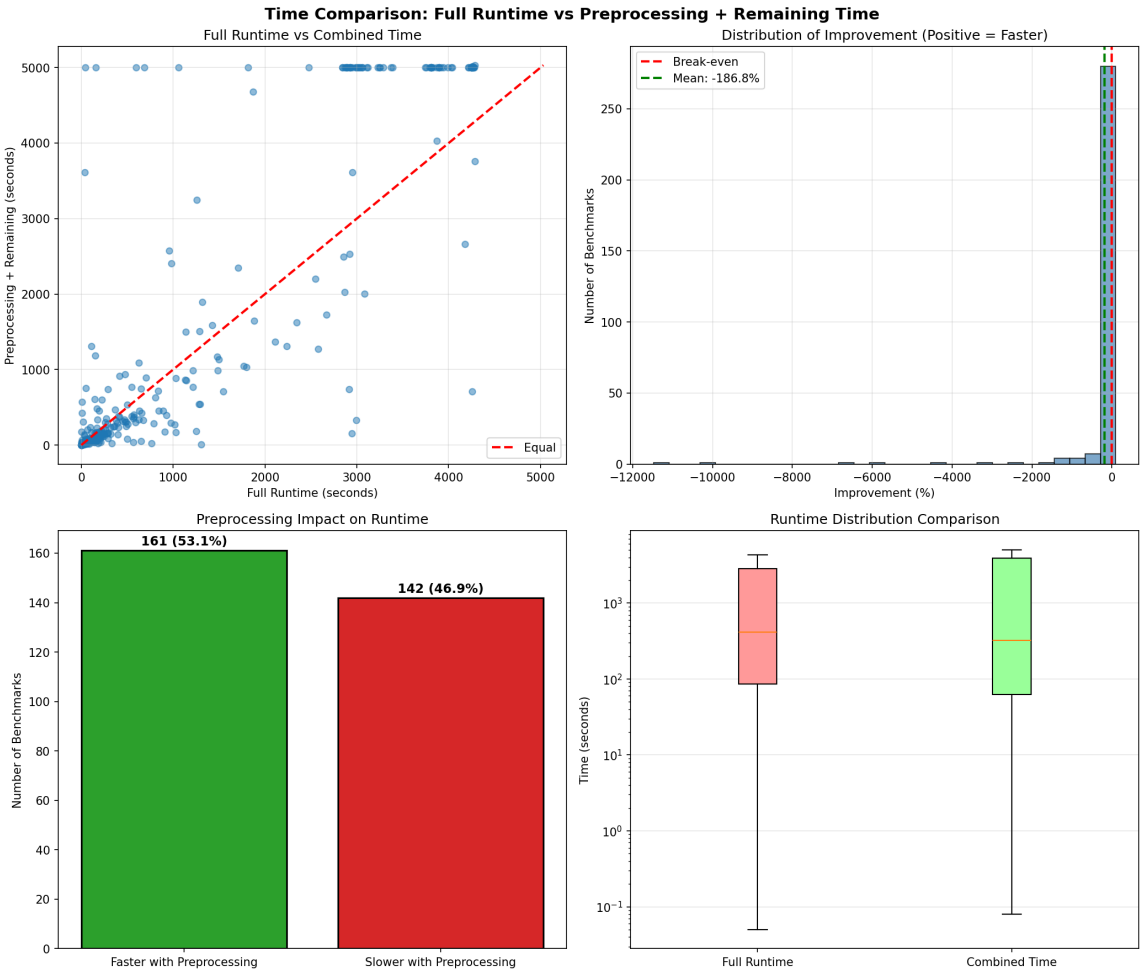


Figure 15: Ergebnisse von Pre100k

Kennzahl	Wert
Anzahl Benchmarks	303
Durchschnittliche Full Runtime	1252,69 Sekunden (∼20,9 Minuten)
Durchschnittliche Preprocessing-Zeit	2,17 Sekunden
Durchschnittliche verbleibende Zeit	1576,17 Sekunden (∼26,3 Minuten)
Durchschnittliche kombinierte Zeit	1578,34 Sekunden (∼26,3 Minuten)
Durchschnittliche Verbesserung	$-325,65$ Sekunden ($-186,8\%$)

Table 22: Zeitvergleich: Full Runtime vs. Preprocessing + Lösung (Pre100k)

Bei der Vereinfachung von pre100k liegt die Verteilung bei 161 Benchamrks die sich verbessert habe (53,1%) wohingegen 142 Benchmarks sich verschlechtert habe (46,9%)

Beste Verbesserungen (Bispiele):

Benchmark	Full Runtime	Kombinierte Zeit	Verbesserung
Ptn-7824-b18	1303,71s	4,70s	+99,6%
battleship-14-26-sat	2,01s	0,08s	+96,0%

Table 23: Beispiele für beste Verbesserungen durch Preprocessing (Pre100k)

Schlechteste Verschlechterungen (Beispiele):

Benchmark	Full Runtime	Kombinierte Zeit	Verschlechterung
EDP3-11000	43,22s	5001,21s	-11.471%
g2-slp-synthesis-aes-top30	35,85s	3610,10s	-9.970%

Table 24: Beispiele für schlechteste Verschlechterungen durch Preprocessing (Pre100k)

4.5.3 Wichtige Erkenntnisse

- **Preprocessing-Zeit**: Pre100k benötigt im Durchschnitt 0,77 Sekunden mehr Vorverarbeitungszeit als Pre10k (2,17s vs. 1,40s). Dies ist erwartungsgemäß, da mehr Simplifikationsschritte durchgeführt werden.

- **Verbleibende Lösungszeit**: Pre100k hat eine minimal höhere verbleibende Lösungszeit (+5,33 Sekunden im Durchschnitt). Dies könnte darauf hindeuten, dass aggressiveres Preprocessing nicht immer zu einer einfacheren Formel führt.

- **Gesamtleistung**: Beide Preprocessing-Stufen schneiden im Durchschnitt schlechter ab als die vollständige Lösung ohne Vorverarbeitung (negative durchschnittliche Verbesserung). Dies ist auf extreme Ausreißer zurückzuführen, bei denen das Preprocessing zu einer massiven Verschlechterung führt.

- **Erfolgsrate**: Pre100k hat eine leicht höhere Erfolgsrate (53,1% schneller vs. 52,4%). Der Unterschied ist jedoch minimal.

- **Extreme Fälle**: Beide Stufen zeigen ähnliche Muster bei extremen Verbesserun-

gen und Verschlechterungen. Besonders auffällig sind Fälle wie `1-ZC-512-K-60`, bei dem Pre10k die Lösungszeit von 0,05 Sekunden auf 341,63 Sekunden verlängert.

4.5.4 Interpretation

Warum ist die durchschnittliche Verbesserung negativ?

Die negativen Durchschnittswerte (Pre10k: -3619%, Pre100k: -186,8%) sind nicht als wörtliche prozentuale Verbesserungen zu interpretieren. Sie entstehen durch extreme Ausreißer, bei denen sehr kurze Full Runtimes (unter 1 Sekunde) durch Preprocessing auf Zeiten um 5000 Sekunden ansteigen. Diese Ausreißer verzerren den Durchschnitt stark.

Beispiel: 1-ZC-512-K-60 (Full Runtime: 0,05s $\rightarrow$ Pre10k: 341,63s) allein verursacht eine scheinbare Verschlechterung von über 600.000%.

Somit soll man Preprocessing nicht als Allheilmittel betrachten, da es in etwas der Hälfte der Fälle die bearbeitungszeit verschlechtert. Des weiteren sieht man das es keinen großen Unterschied macht wie stark man die Benchamrks vereinfacht. Preprocessing sollte man meiner Meinung dann nur anwenden auf Benchmarks für die es tatsächlich auch wirkt.

4.5.5 Fazit

Die Untersuchung zeigt, dass sowohl Pre10k (10.000 Simplifikationsschritte) als auch Pre100k (100.000 Simplifikationsschritte) in etwa der Hälfte der Fälle (52-53%) zu einer Beschleunigung der Lösungszeit führen. Die durchschnittliche Preprocessing-Zeit ist mit 1,4 bis 2,2 Sekunden vernachlässigbar gering. Allerdings kann Preprocessing in einigen Fällen zu extremen Verlangsamungen führen (bis zu Faktor 10.000), was die durchschnittliche Verbesserung verzerrt. Ein signifikanter Vorteil von Pre100k gegenüber Pre10k konnte ich daher nicht festgestellen. Für die praktische Anwendung wird daher eine selektive, instanzabhängige Aktivierung des Preprocessings empfohlen.

5 Conclusion

In dieser Arbeit wurde untersucht, ob und wie gut SATZilla – ein auf maschinellem Lernen basierendes Algorithmus-Portfolio – für die Auswahl zwischen verschiedenen Konfigurationen des modernen CDCL-SAT-Solvers Kissat eingesetzt werden kann. Die Experimente umfassten drei Hauptphasen: (1) Validierung der Pipeline mit künstlichen ("Fake")-Solvern, (2) Training und Evaluation auf sechs realen Kissat-Varianten mit den ursprünglichen Benchmarks sowie (3) Analyse der Auswirkungen von Vorverarbeitung auf die Vorhersagen und Laufzeiten.

Erstens bestätigt die erfolgreiche Validierung mit Fake-Solvern, dass die SATZilla-Pipeline – von der Feature-Extraktion über das Training bis zur Vorhersage – korrekt implementiert ist. Die hohe Genauigkeit und die plausible Features beweisen, dass das Verfahren lernen kann.

Zweitens zeigt die Anwendung auf sechs echte Kissat-Varianten, dass SATZilla auch bei komplexen, realen Solvern nutzbare Vorhersagen liefert. Mit einer Top-3-Genauigkeit von 85 % und einem durchschnittlichen Speedup von 1,47 gegenüber dem Standard-Solver ist SATZilla nicht nur besser als ein Zufallsverfahren, sondern auch praktisch nützlich für die portfoliogestützte SAT-Lösung. Die Analyse der Features offenbart zudem, welche strukturellen Eigenschaften einer Instanz für die Leistungsunterschiede der Kissat-Varianten verantwortlich sind – ein Erkenntnisgewinn, der über die reine Solver-Auswahl hinausgeht.

Drittens belegt die Untersuchung der Vorverarbeitung, dass SATZillas Vorhersagen nicht vorverarbeitungsinvariant sind. Die Änderungen der Solver-Empfehlungen sind eine direkte Folge der tatsächlichen strukturellen Veränderungen der Formeln, nicht willkürlich. Die Dosis-Wirkungs-Beziehung (mehr Vorverarbeitung $\rightarrow$ mehr Änderungen) validiert die Sensitivität des Feature-Sets. Für die praktische Anwendung bedeutet dies, dass SATZilla in diesem Fall Vorhersagen macht die Zutreffender sein könnten.

Viertens zeigt der Laufzeitvergleich, dass Vorverarbeitung kein Allheilmittel ist: Sie beschleunigt zwar etwa die Hälfte der Instanzen, kann in Einzelfällen aber zu extremen Verlangsamungen führen. Ein signifikanter Vorteil von 100.000 gegenüber 10.000 Vereinfachungsschritten konnte nicht festgestellt werden.

Abschließend möchte ich sagen das mir die Arbeit sehr gefallen hat und ich viel über ein neues Thema lernen konnte. Besonders spannend fand ich die Arbeit an der Forschung von Vorhersagen und die anschließende Auswertung der Daten.

5.1 Technische Daten

Die Arbeit wurde von meinem Windows 11 Heim PC erstellt und mit WSl, damit diesem konnte ich mich mit einer ssh verbindung am PC der Technischen Fakultät anmelden, dies war von großem Vorteil da ich breits im meinem Bachelorprojekt mehr RAM für meine brechnungen brauchte. Tmux erwies sich als nützlich um Programme über einen längeren Zeitraum laufen lassen zu können.

Das Programmiern der Skripte wurde mit Python 3.9 geschrieben mit folgenden Bibliotheken: numpy, pandas, scikit-learn (random forest), matplotlib (zum erstellen der Diagramme und Visualierungen), seaborn, joblib, plotly, tqdm, requests. Ausgeführt wurden die Skripte mit Conda.

List of Figures

1 Kissat SAT Solver [3] 6
2 Testgenauigkeit der verschiedenen Strategien 17
3 Vergleich mit Random Selection 17
4 Zusammenfassung der Besten Solver 18
5 Vergleich der drei besten Solver mit Zufallsprinzip 18
6 Für jeden solver Top 3 Vorhersagen 19
7 Wichtigste Features für die Vorhersagen 21
8 Vorhersagen Genauigkeit nach Solver 27
9 Vorhersagen Korrektheit nach Solver 27
10 Solver-Verteilung für Original-, Pre10k- und Pre100k-Benchmarks . . . 37
11 Anzahl der Solver Änderungen für Original-, Pre10k- und Pre100k-Benchmarks 39
12 Änderung der jeweiligen Solver für Original-, Pre10k- und Pre100k-Benchmarks 41
13 Vergleich der Top-15 Features von Pre10k- und Pre100k-Benchmarks . 45
14 Ergebnisse von Pre10k 49
15 Ergebnisse von Pre100k 51

List of Tables

1 Kissat-Konfigurationsvarianten für diese Arbeit 9
2 SAT Instance Features Categories 11
3 Testergebnisse für verschiedene Auswahlstrategien 14
4 Testergebnisse der wichtigsten Features 14
5 Speedup-Kennzahlen von Satzilla gegenüber kissat_default 20
6 VCG Klausen Features and ihre wichtigkeit 22
7 Preprocessing Features and ihre wichtigkeit 23
8 Preprocessing Features and ihre wichtigkeit 24
9 Klausel länge Features and ihre wichtigkeit 24
10 Basic size Features and ihre wichtigkeit 25
11 Feature Kategorien und ihre wichtigkeit 26
12 Unterschied der Vorhersage mit LP-Features 29
13 Feature Extraction Statistics 35
14 Random Forest Konfigurationsparameter 35
15 Solver Selection Distribution (%) 38
16 Vorhersageänderungen zwischen verschiedenen Vorverarbeitungsstufen 39
17 Häufigste Änderungen (Original → Pre10k) 41
18 Vergleich der Feature-Wichtigkeiten: Pre10k vs Pre100k 46
19 Zeitvergleich: Full Runtime vs. Preprocessing + Lösung 49
20 Beispiele für beste Verbesserungen durch Preprocessing 50
21 Beispiele für schlechteste Verschlechterungen durch Preprocessing . . 50
22 Zeitvergleich: Full Runtime vs. Preprocessing + Lösung (Pre100k) . . . 51
23 Beispiele für beste Verbesserungen durch Preprocessing (Pre100k) . . 52
24 Beispiele für schlechteste Verschlechterungen durch Preprocessing (Pre100k) 52

References

[1] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. Cadical, kissat, paracooba, plingeling and treengeling entering the sat competition 2020. In *Proc. of SAT Competition 2020 - Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 50–53, Helsinki, 2020. University of Helsinki.

[2] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158, New York, NY, USA, 1971. ACM.

[3] Institute for Formal Models and Verification. Kissat sat solver. `https://fmv.jku.at/kissat/`, 2020. Abgerufen am 01.05.2026.

[4] Eugene Nudelman, Alex Devkar, Yoav Shoham, Kevin Leyton-Brown, and Holger H. Hoos. Satzilla : An algorithm portfolio for sat. In *Proceedings of the 19th National Conference on Artificial Intelligence (AAAI-04)*. AAAI Press, 2004.

[5] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Satzilla: Portfolio-based algorithm selection for sat. *Journal of Artificial Intelligence Research (JAIR)*, 32:565–606, 2008.